

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA VẬT LÝ



NHÓM 11

PHƯƠNG PHÁP NHẬN DIỆN VẬT CẢN VÀ ƯỚC
LƯỢNG KHOẢNG CÁCH ĐỂ HỖ TRỢ NGƯỜI
KHIẾM THỊ

Đề tài môn Học Máy
Ngành Kỹ thuật Điện tử và Tin học
(Chương trình đào tạo chuẩn)

Hà Nội - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA VẬT LÝ



NHÓM 11

PHƯƠNG PHÁP NHẬN DIỆN VẬT CẢN VÀ ƯỚC
LƯỢNG KHOẢNG CÁCH ĐỂ HỖ TRỢ NGƯỜI
KHIẾM THỊ

Đề tài môn Học Máy
Ngành Kỹ thuật Điện tử và Tin học
(Chương trình đào tạo chuẩn)

Giảng viên hướng dẫn: TS. Nguyễn Tiến Cường

CN. Vi Anh Quân

Hà Nội - 2026

Lời cảm ơn

Trong quá trình thực hiện môn học Học máy (Machine Learning) để hoàn thành bài báo cáo này, nhóm em đã nhận được sự hướng dẫn tận tình, sự quan tâm sâu sắc và những hỗ trợ quý báu từ thầy Nguyễn Tiến Cường và thầy Vi Anh Quân. Nhóm em xin gửi lời cảm ơn chân thành và sâu sắc đến các thầy vì đã luôn đồng hành, định hướng và hỗ trợ em trong suốt quá trình học tập cũng như triển khai đề tài. Trong suốt quá trình thực hiện, em đã được tiếp cận nhiều kiến thức quan trọng liên quan đến các mô hình học sâu, xử lý dữ liệu và ứng dụng trí tuệ nhân tạo trong thực tế. Những góp ý chuyên môn của các thầy không chỉ giúp em hoàn thiện nội dung bài báo cáo mà còn giúp em hiểu rõ hơn về tư duy giải quyết vấn đề trong lĩnh vực học máy.

Mặc dù đã cố gắng hoàn thiện bài báo cáo một cách tốt nhất trong khả năng của mình, nhưng do hạn chế về kiến thức và kinh nghiệm thực tiễn nên không thể tránh khỏi những thiếu sót. Nhóm em rất mong nhận được những ý kiến đóng góp, nhận xét và chỉ dẫn thêm từ các thầy để có thể tiếp tục hoàn thiện báo cáo trong thời gian tới.

Cuối cùng, nhóm em xin kính chúc thầy Nguyễn Tiến Cường và thầy Vi Anh Quân luôn mạnh khỏe, hạnh phúc và đạt được nhiều thành công trong sự nghiệp giảng dạy và nghiên cứu khoa học.

Nhóm em xin chân thành cảm ơn các thầy!

Nhóm sinh viên thực hiện

Lớp Kỹ thuật Điện tử và Tin học K68 – Khoa Vật lý

STT	Họ và tên – Mã sinh viên	Tỷ lệ đóng góp (%)
1	Nguyễn Tiến Dũng – 23001585	40
2	Hà Việt Anh – 23001575	35
3	Cao Văn Đạt – 23001591	25

Mục lục

Lời cảm ơn	i
Danh sách hình vẽ	v
Danh sách bảng	vi
Danh sách tên viết tắt	vii
MỞ ĐẦU	1
1 TỔNG QUAN ĐỀ TÀI	3
1.1 Giới thiệu bài toán	3
1.2 Các phương pháp nghiên cứu trước đây	4
1.2.1 Phương pháp Faster R-CNN	5
1.2.2 Phương pháp SSD	6
1.2.3 Phương pháp RetinaNet	8
1.3 Phương pháp đề xuất	9
2 CƠ SỞ LÝ THUYẾT	11
2.1 Mô hình phát hiện đối tượng YOLO	11
2.1.1 Mô hình YOLO26	12
2.2 Multi-Object Tracking	16
2.2.1 Phương pháp Centroid Tracking	16
2.3 MIDAS	18
2.3.1 Bài toán ước lượng độ sâu	18
2.3.2 Kiến trúc tổng quan của mô hình MiDaS	20
2.4 gTTS	21
2.4.1 Khái niệm	21
2.4.2 Cơ chế hoạt động	22
2.4.3 Các tính năng chính của thư viện	23
3 PHƯƠNG PHÁP	24
3.1 Kiến trúc bài toán	24
3.2 Dữ liệu thực nghiệm	27
3.2.1 Đặc điểm bộ dữ liệu	28
3.2.2 Tiền xử lý dữ liệu	30
3.2.3 Đặc trưng dữ liệu	31
	iii

3.2.4	Phân chia dữ liệu phục vụ huấn luyện	32
3.3	Hàm mất mát	33
3.3.1	Hàm mất mát của YOLO26	33
3.3.2	Hàm mất mát của MiDAS	35
3.4	Huấn luyện mô hình	37
3.5	Nguyên lý hoạt động của các phương pháp xử lý hỗ trợ điều hướng . . .	39
3.5.1	Phương pháp theo dõi đa đối tượng ByteTrack	39
3.5.2	Phương pháp ước tính chiều sâu và phân vùng khoảng cách . . .	43
3.5.3	Phương pháp tổng hợp ngôn ngữ và cơ chế dự phòng đa tầng . .	47
4	THỰC NGHIỆM VÀ ĐÁNH GIÁ	51
4.1	Kết quả thực nghiệm	51
4.1.1	Đánh giá mô hình	51
4.1.2	Phân tích và đánh giá kết quả	53
5	TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN	56
5.1	Tổng kết và đánh giá hệ thống luận chứng	56
5.2	Hạn chế và hướng phát triển	58
	Tài liệu tham khảo	60
A	Các chỉ số đánh giá	62
B	Liệt kê source code	66

Danh sách hình vẽ

1.1	Kiến trúc của phương pháp Faster R-CNN [6].	5
1.2	Kiến trúc của phương pháp SSD [7].	6
1.3	Cơ chế hoạt động của phương pháp SSD [8].	7
1.4	Kiến trúc phương pháp RetinaNet [10].	8
1.5	Pipeline phương pháp đề xuất.	9
2.1	Mô hình mạng CNN được sử dụng trong YOLO [15]	11
2.2	Kiến trúc tổng quan YOLO26	13
2.3	Kiến trúc NMS-Free [3]	13
2.4	Loại bỏ DFL [3]	14
2.5	Tối ưu cho vật thể nhỏ [3]	15
2.6	Tối ưu hóa MuSGD [3]	15
2.7	Kiến trúc Centroid Tracking.	17
2.8	Hình ảnh minh họa [11]	19
2.9	Kiến trúc mô hình MIDAS [11]	20
2.10	Sơ đồ khối cơ chế hoạt động của hệ thống gTTS	22
3.1	Kiến trúc tổng thể của mô hình	24
3.2	Data Acquisition Layer	25
3.3	Backend Layer	25
3.4	Perception Layer	26
3.5	Decision Layer	27
3.6	Application Layer	27
3.7	Hình ảnh minh họa bộ dữ liệu WOTR [20]	29
3.8	Hình ảnh dữ liệu được thu thập	29
3.9	Gán nhãn dữ liệu	31
A.1	Ma trận nhầm lẫn theo từng thuộc tính	64

Danh sách bảng

3.1	Thống kê số lượng nhãn chi tiết theo từng lớp đối tượng.	32
3.2	Thống kê phân chia bộ dữ liệu.	33
3.3	Cấu hình thông số huấn luyện mô hình YOLO26.	38
4.1	Kết quả thực nghiệm chi tiết của mô hình YOLO26 trên các lớp đối tượng.	52
A.1	Ma trận nhầm lẫn kết quả phân lớp khoảng cách cảnh báo của MiDaS.	65

Danh sách tên viết tắt

AI	Artificial Intelligence (Trí tuệ nhân tạo)
YOLO	You Only Look Once
Faster R-CNN	Faster Region-based Convolutional Neural Network
RPN	Region Proposal Network
SSD	Single Shot MultiBox Detector
FPN	Feature Pyramid Network
PAN	Path Aggregation Network
RetinaNet	Retina Network
MiDaS	Monocular Depth Estimation
TTS	Text-to-Speech
CNN	Convolutional Neural Network
TAL	Task-Aligned Learning
NMS	Non-Maximum Suppression
DFL	Distribution Focal Loss
ONNX	Open Neural Network Exchange
STAL	Small-Target-Aware Label Assignment
SGD	Stochastic Gradient Descent
MuSGD	Momentum-based Stochastic Gradient Descent
MOT	Multi-Object Tracking
gTTS	Google Text-to-Speech
CLI	Command Line Interface
API	Application Programming Interface
HTTP	HyperText Transfer Protocol
NLP	Natural Language Processing
CPU	Central Processing Unit
CIoU	Complete Intersection over Union
IoU	Intersection over Union
MAE	Mean Absolute Error
MSE	Mean Squared Error
RMSE	Root Mean Squared Error
GPU	Graphics Processing Unit
MD5	Message-Digest Algorithm 5

MỞ ĐẦU

Trong bối cảnh công nghệ trí tuệ nhân tạo (AI) và thị giác máy tính đang phát triển mạnh mẽ, các ứng dụng hỗ trợ con người trong đời sống ngày càng được quan tâm và nghiên cứu rộng rãi. Đặc biệt, đối với người khiếm thị, việc di chuyển và nhận biết môi trường xung quanh luôn là một thách thức lớn do hạn chế về khả năng quan sát. Những khó khăn trong việc phát hiện vật cản, xác định khoảng cách hay nhận biết nguy hiểm có thể ảnh hưởng trực tiếp đến sự an toàn và khả năng sinh hoạt độc lập của người khiếm thị trong cuộc sống hàng ngày.

Hiện nay, nhiều giải pháp hỗ trợ người khiếm thị đã được nghiên cứu và phát triển, từ các thiết bị cảm biến siêu âm truyền thống cho đến các hệ thống ứng dụng trí tuệ nhân tạo và học sâu (Deep Learning). Trong đó, các phương pháp sử dụng thị giác máy tính kết hợp với nhận diện đối tượng và ước lượng khoảng cách đang cho thấy nhiều tiềm năng nhờ khả năng xử lý thông tin trực quan nhanh chóng, chính xác và có thể hoạt động trong thời gian thực. Tuy nhiên, việc xây dựng các phương pháp có độ chính xác cao, khả năng thích nghi tốt với môi trường thực tế và đáp ứng yêu cầu xử lý thời gian thực vẫn còn là một bài toán cần được nghiên cứu và cải thiện.

Xuất phát từ thực tế đó, nhóm em lựa chọn đề tài “Phương pháp nhận diện vật cản và ước lượng khoảng cách để hỗ trợ người khiếm thị”. Đề tài tập trung nghiên cứu các phương pháp nhận diện vật cản dựa trên mô hình học sâu kết hợp với kỹ thuật ước lượng khoảng cách nhằm hỗ trợ người khiếm thị trong quá trình di chuyển. Thông qua việc phân tích, xây dựng và đánh giá các mô hình phù hợp, đề tài hướng tới việc nâng cao khả năng nhận biết môi trường xung quanh, hỗ trợ cảnh báo vật cản và tăng độ an toàn cho người khiếm thị.

Trong báo cáo này, nhóm em trình bày quá trình nghiên cứu lý thuyết, xây dựng

phương pháp, triển khai mô hình và đánh giá kết quả thực nghiệm cho bài toán nhận diện vật cản và ước lượng khoảng cách nhằm hỗ trợ người khiếm thị. Báo cáo tập trung phân tích cách thức hoạt động của mô hình, khả năng nhận diện vật cản cũng như hiệu quả ước lượng khoảng cách trong các điều kiện thực tế khác nhau. Qua đó, đề tài hướng tới việc đề xuất một phương pháp hỗ trợ người khiếm thị trong quá trình di chuyển, góp phần nâng cao tính an toàn và khả năng ứng dụng của các giải pháp hỗ trợ thông minh trong thực tế.

Chương 1 TỔNG QUAN ĐỀ TÀI

1.1 Giới thiệu bài toán

Trong kỷ nguyên của trí tuệ nhân tạo và các hệ thống tự hành, bài toán nhận diện vật cản và ước lượng khoảng cách đã trở thành một trong những hướng nghiên cứu trọng tâm và tiên phong của lĩnh vực Thị giác máy tính (Computer Vision). Ý nghĩa của bài toán này không chỉ dừng lại ở việc tối ưu hóa công nghệ, mà còn mang giá trị nhân văn sâu sắc khi được ứng dụng để xây dựng các hệ thống điều hướng thông minh, hỗ trợ người khiếm thị tái hòa nhập cộng đồng [1]. Đối với người khiếm thị, việc di chuyển trong môi trường thực tế là một thách thức lớn khi xung quanh luôn tồn tại các mối nguy cơ chuyển động liên tục và bất ngờ như người đi bộ, xe máy, ô tô, hoặc các chướng ngại vật tĩnh như cột điện, biển báo, bàn ghế. Một hệ thống trợ thính hoặc trợ thị thông minh không chỉ cần đóng vai trò như một đôi mắt thứ hai để nhận biết môi trường, mà còn phải là một bộ não có khả năng phân tích không gian trong thời gian thực, từ đó đưa ra những chỉ dẫn an toàn và kịp thời cho người dùng [2].

Để giải quyết trọn vẹn bài toán hỗ trợ điều hướng, hệ thống không thể dựa vào một kỹ thuật đơn lẻ mà phải phối hợp nhịp nhàng một chuỗi các tác vụ phức tạp theo trình tự tuyến tính. Đầu tiên là tác vụ nhận diện đối tượng (Object Detection) nhằm phân tích các khung hình thu nhận từ camera để xác định chính xác danh tính và vị trí của các vật cản xuất hiện trong tầm nhìn. Tiếp theo, tác vụ theo dõi đối tượng (Object Tracking) được thực hiện để duy trì thông tin định danh của từng vật thể xuyên suốt các khung hình liên tiếp, giúp hệ thống hiểu được quỹ đạo và hướng di chuyển của vật cản, đồng thời phân biệt giữa chướng ngại vật tĩnh và động. Cuối cùng, tác vụ ước lượng độ sâu (Depth Estimation) tiến hành đo lường khoảng cách tương đối từ camera đến vật cản, cung cấp dữ liệu số học nền tảng để đánh giá mức độ rủi ro va chạm.

Mặc dù lý thuyết về các mô hình độc lập đã đạt được nhiều bước tiến, việc tích hợp chúng vào một hệ thống hỗ trợ người khiếm thị trong môi trường thực tế vẫn đối mặt với những rào cản kỹ thuật lớn. Sự biến động của môi trường như điều kiện ánh sáng thay đổi thất thường, mật độ giao thông đông đúc, và hiện tượng các vật cản che khuất lẫn nhau dễ làm giảm độ chính xác của mô hình nhận diện. Bên cạnh đó, hệ thống phục vụ người đi lại luôn yêu cầu tốc độ phản hồi cực cao dưới áp lực xử lý thời gian thực, bởi bất kỳ sự trễ nào trong quá trình tính toán đều có thể dẫn đến nguy cơ va chạm thực tế. Ngoài ra, việc lồng ghép nhiều mô hình học sâu (Deep Learning) vào cùng một hệ thống sẽ gây áp lực lớn lên tài nguyên phần cứng, đòi hỏi phải giải quyết triệt để bài toán tối ưu hóa tốc độ xử lý và đồng bộ hóa dữ liệu giữa các luồng thuật toán.

Để vượt qua các thách thức trên, đề tài này đề xuất một giải pháp toàn diện sử dụng mô hình YOLO26 làm kiến trúc cốt lõi cho tác vụ nhận diện vật cản nhờ tốc độ xử lý vượt trội và khả năng nhận diện đa lớp vật thể chính xác [3]. Song song với đó, thuật toán ByteTrack được tích hợp để theo dõi đối tượng xuyên khung hình, duy trì quỹ đạo chuyển động ổn định ngay cả khi vật cản bị che khuất tạm thời [4]. Đối với việc tính toán khoảng cách, mô hình MiDaS được ứng dụng để ước lượng độ sâu từ ảnh đơn, cho phép tối ưu chi phí phần cứng mà không cần đến các hệ thống camera kép hay cảm biến đắt đỏ [5]. Cuối cùng, các luồng dữ liệu này được tổng hợp thông qua một thuật toán phân tích mức độ nguy hiểm để phát ra cảnh báo bằng giọng nói tiếng Việt trực quan.

1.2 Các phương pháp nghiên cứu trước đây

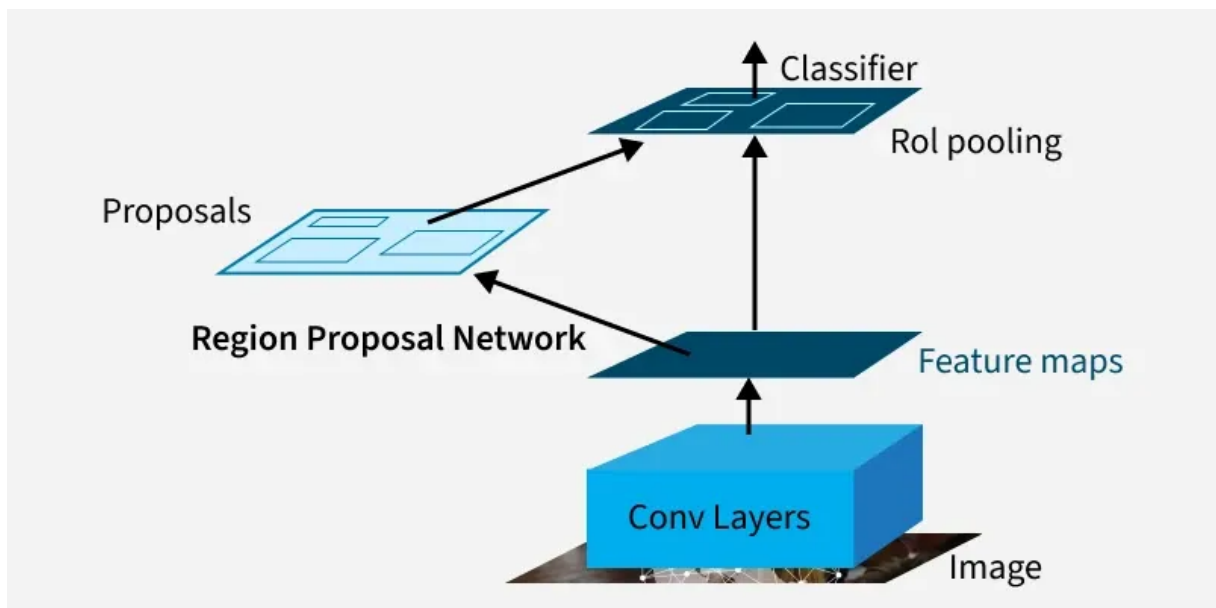
Các nghiên cứu trước đây chủ yếu tập trung vào bài toán phát hiện đối tượng (Object Detection) nhằm xác định đồng thời vị trí và nhãn phân loại của các vật thể trong môi trường. Dựa trên cấu trúc kiến trúc mạng, các phương pháp này được chia thành hai nhóm tiếp cận chính bao gồm mô hình phát hiện đối tượng hai giai đoạn (Two-stage detector) và mô hình phát hiện đối tượng một giai đoạn (One-stage detector). Việc phân

tích và đánh giá chi tiết các phương pháp điển hình này là cơ sở lý thuyết quan trọng để định hình giải pháp tối ưu cho hệ thống hỗ trợ người khiếm thị.

1.2.1 Phương pháp *Faster R-CNN*

Faster R-CNN là mô hình đại diện tiêu biểu và thành công nhất trong nhóm kiến trúc bộ dò hai giai đoạn [6]. Mô hình này ra đời nhằm khắc phục nhược điểm về tốc độ của các phiên bản tiền nhiệm bằng cách loại bỏ thuật toán trích xuất vùng đề xuất truyền thống mang tính thủ công và thay thế bằng một mạng thần kinh tích chập đồng bộ.

Quy trình hoạt động của Faster R-CNN được chia làm hai giai đoạn rõ rệt. Trong giai đoạn đầu tiên, hình ảnh đầu vào được xử lý qua một mạng nền tảng để trích xuất ra các bản đồ đặc trưng, sau đó một mạng đề xuất vùng chuyên biệt có tên gọi là Region Proposal Network sẽ tiến hành quét qua bản đồ này để dự đoán xác suất chứa đối tượng và đưa ra các khung hình ứng viên sơ bộ. Sang giai đoạn thứ hai, các vùng đề xuất được chuẩn hóa về một kích thước cố định thông qua lớp RoI Pooling rồi chuyển đến các lớp kết nối đầy đủ để thực hiện song song hai nhiệm vụ là phân loại chính xác nhãn của đối tượng và tính chỉnh tối ưu tọa độ khung bao cuối cùng [6].

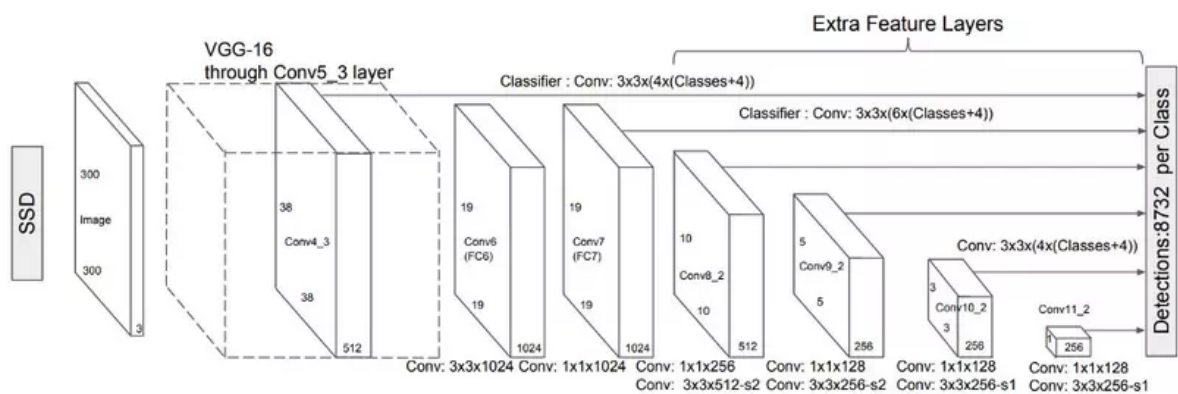


Hình 1.1: Kiến trúc của phương pháp Faster R-CNN [6].

Nhờ cơ chế tách biệt hai giai đoạn mang tính tuần tự này, Faster R-CNN sở hữu ưu điểm vượt trội về độ chính xác khi nhận diện. Mô hình đặc biệt xuất sắc trong việc phát hiện các vật thể có kích thước nhỏ, cấu trúc phức tạp hoặc các đối tượng bị che khuất một phần trong không gian. Tuy nhiên, nhược điểm lớn nhất của Faster R-CNN lại nằm ở tốc độ suy luận do việc phải xử lý hàng ngàn vùng đề xuất tiêu tốn lượng tài nguyên tính toán cực kỳ lớn. Chính sự hạn chế về mặt tốc độ này khiến mô hình không đạt được tính khả thi khi triển khai trên các thiết bị nhúng cầm tay hoặc các hệ thống yêu cầu phản hồi thời gian thực tức thời như thiết bị hỗ trợ người khiếm thị di chuyển.

1.2.2 Phương pháp SSD

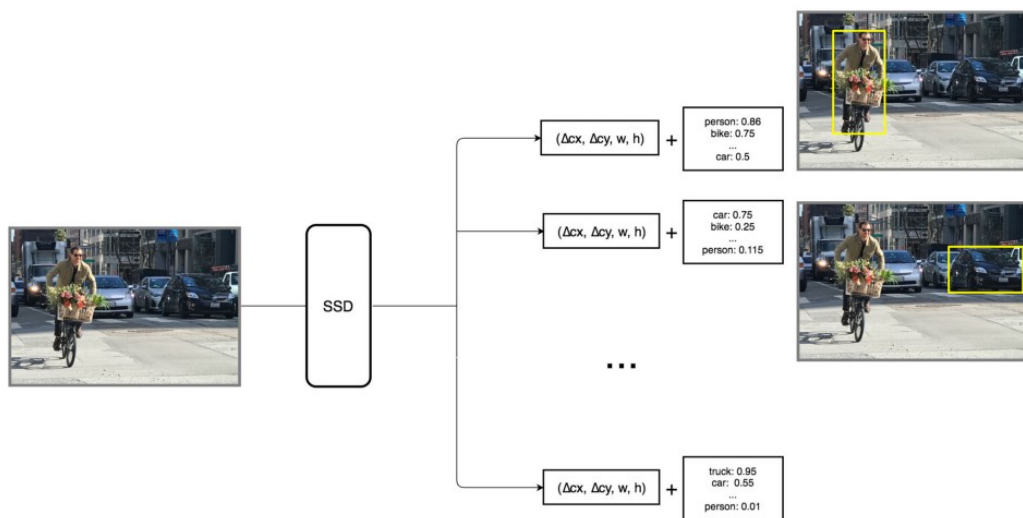
Mô hình SSD (Single Shot MultiBox Detector) được giới thiệu nhằm mục tiêu tối ưu hóa tốc độ xử lý của kiến trúc một giai đoạn nhưng vẫn cố gắng duy trì độ chính xác tiệm cận với các bộ dò hai giai đoạn phức tạp [7]. Cấu trúc và cơ chế hoạt động cốt lõi của SSD được xây dựng dựa trên sự kết hợp của ba thành phần kỹ thuật nền tảng bao gồm mạng nền tảng cơ sở, hệ thống bản đồ đặc trưng đa tỷ lệ và cơ chế khung neo mặc định.



Hình 1.2: Kiến trúc của phương pháp SSD [7].

Mạng cơ sở của SSD thường sử dụng kiến trúc VGG-16 hoặc MobileNet đã được loại bỏ các tầng kết nối đầy đủ (Fully Connected Layers) nhằm trích xuất các đặc trưng thô từ hình ảnh đầu vào với chi phí tính toán thấp. Điểm cải tiến lớn nhất của SSD so với

các mô hình cùng thời kỳ nằm ở việc bổ sung một chuỗi các tầng tích chập phụ với kích thước giảm dần một cách lũy tiến ngay sau mạng nền tảng, tạo ra một hệ thống các bản đồ đặc trưng đa tỷ lệ (Multi-scale Feature Maps). Quy trình này cho phép mô hình thực hiện dự đoán độc lập trên nhiều tầng mạng khác nhau [7]. Cụ thể, các tầng bản đồ kích thước lớn ở phía nông có độ phân giải không gian cao sẽ chịu trách nhiệm phát hiện các vật thể kích thước nhỏ, trong khi các tầng bản đồ kích thước nhỏ ở phía sâu chứa nhiều hàm lượng thông tin ngữ nghĩa cao cấp sẽ chịu trách nhiệm cấu trúc và phát hiện các vật thể có kích thước lớn hơn.



Hình 1.3: Cơ chế hoạt động của phương pháp SSD [8].

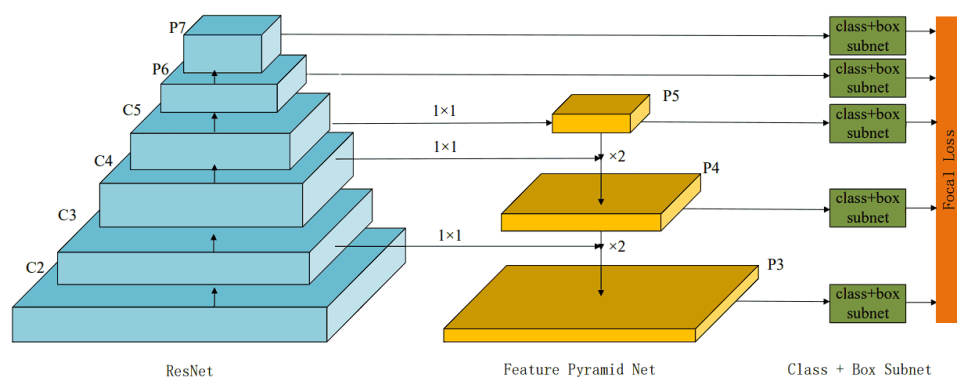
Tại mỗi ô lưới trên từng bản đồ đặc trưng đa tỷ lệ này, SSD phân bố một tập hợp các khung neo mặc định (Default Boxes) với nhiều tỷ lệ khía cạnh và kích thước hình học khác nhau. Trong quá trình suy luận, mô hình sẽ tính toán trực tiếp độ lệch tọa độ của khung bao thực tế so với các khung neo mặc định này, đồng thời dự đoán điểm số tin cậy cho từng lớp đối tượng cụ thể. Cuối cùng, hệ thống áp dụng thuật toán triệt tiêu cực đại không giao nhau (Non-Maximum Suppression) để lọc sạch các khung bao chồng chéo và giữ lại kết quả tối ưu nhất [8].

Mặc dù SSD (Single Shot MultiBox Detector) là một mô hình phát hiện đối tượng thời gian thực có tốc độ xử lý cao, phương pháp này chưa thực sự phù hợp với hệ thống

hỗ trợ người khiếm thị. Do không sử dụng các cơ chế hợp nhất đặc trưng đa tỷ lệ hiện đại như FPN hay PAN, SSD gặp hạn chế trong việc phát hiện các vật thể nhỏ hoặc ở khoảng cách xa, làm tăng nguy cơ bỏ sót những chương ngại vật quan trọng như biển báo, cột điện hoặc vật cản trên mặt đường.

1.2.3 Phương pháp RetinaNet

RetinaNet là một mô hình một giai đoạn mạng tính đột phá được đề xuất nhằm giải quyết triệt để bài toán mất cân bằng nghiêm trọng giữa lớp nền và lớp đối tượng, một điểm yếu cốt lõi khiến các bộ dò một giai đoạn trước đó có độ chính xác thấp hơn bộ dò hai giai đoạn [9]. Kiến trúc của RetinaNet là sự kết hợp tinh tế giữa mạng nền tảng ResNet, mạng kim tự tháp đặc trưng (Feature Pyramid Network) để trích xuất đặc trưng đa quy mô đa tầng, và hai mạng nhánh phụ chuyên biệt chịu trách nhiệm phân loại đối tượng và hồi quy khung bao. Điểm đổi mới quan trọng nhất làm nên sự thành công của RetinaNet là việc giới thiệu một hàm mất mát mới mang tên Focal Loss. Hàm mất mát này tự động hạ thấp trọng số đóng góp của các mẫu dễ nhận diện (chủ yếu là vùng nền trống) và tập trung sự chú ý của mô hình vào các mẫu khó nhận diện (vật cản thực tế), giúp mạng học được các đặc trưng quan trọng một cách hiệu quả hơn.



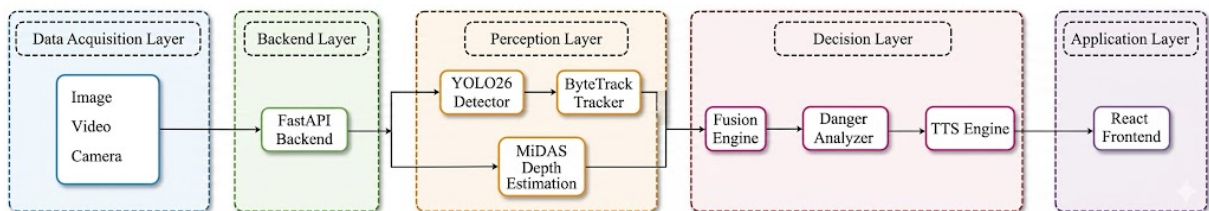
Hình 1.4: Kiến trúc phương pháp RetinaNet [10].

Về mặt ưu điểm, RetinaNet đạt được sự cân bằng lý tưởng khi sở hữu độ chính xác

cao tiệm cận với các mô hình hai giai đoạn phức tạp như Faster R-CNN, nhưng vẫn duy trì được tốc độ xử lý nhanh chóng của một bộ dò một giai đoạn nhờ cấu trúc không cần vùng đề xuất [9]. Điều này giúp mô hình nhận diện tốt các chương ngại vật đa dạng trên đường phố ngay cả trong điều kiện mật độ giao thông đông đúc. Tuy nhiên, nhược điểm của RetinaNet là cấu trúc mạng kim tự tháp đặc trưng đi kèm với cơ chế Focal Loss vẫn đòi hỏi một lượng tài nguyên tính toán nhất định ở giai đoạn huấn luyện và có thể gặp hiện tượng giảm tốc độ nhẹ trên các vi điều khiển công suất thấp nếu không được tối ưu hóa hoặc nén mô hình một cách hợp lý.

1.3 Phương pháp đề xuất

Trong đề tài này, phương pháp đề xuất tập trung vào việc xây dựng phương pháp hỗ trợ điều hướng thông minh dành cho người khiếm thị thông qua việc kết hợp nhiều mô hình trí tuệ nhân tạo trong cùng một pipeline như Hình 1.5. Phương pháp sử dụng camera để thu nhận dữ liệu hình ảnh từ môi trường xung quanh, sau đó thực hiện nhận diện vật cản, theo dõi đối tượng, ước lượng khoảng cách và cảnh báo nguy hiểm bằng giọng nói.



Hình 1.5: Pipeline phương pháp đề xuất.

Đầu tiên, mô hình YOLO26 được sử dụng để thực hiện bài toán nhận diện vật cản [3]. Mô hình có nhiệm vụ phát hiện các đối tượng xuất hiện trong khung hình như người đi bộ, xe máy, ô tô hoặc các chương ngại vật khác. YOLO26 được lựa chọn do có khả năng xử lý nhanh và phù hợp với các ứng dụng thời gian thực. Ngoài ra, mô hình được fine-tune trên tập dữ liệu phù hợp nhằm nâng cao độ chính xác trong môi trường thực tế.

Song song với quá trình nhận diện đối tượng, hệ thống sử dụng mô hình MiDaS để thực hiện bài toán Monocular Depth Estimation [11]. Mô hình này cho phép ước lượng độ sâu và khoảng cách của vật thể chỉ từ một camera đơn mà không cần sử dụng camera stereo hoặc cảm biến chuyên dụng. Kết quả của MiDaS là bản đồ độ sâu (Depth Map), thể hiện mức độ gần hoặc xa của các vật thể trong không gian [11].

Sau khi phát hiện đối tượng, hệ thống sử dụng thuật toán ByteTrack để thực hiện theo dõi đa đối tượng (Multi-Object Tracking) [12]. ByteTrack có nhiệm vụ gán ID cho từng đối tượng và duy trì việc theo dõi xuyên suốt nhiều khung hình liên tiếp. Điều này giúp hệ thống xác định được hướng di chuyển cũng như trạng thái thay đổi của vật cản theo thời gian thực.

Kết quả từ YOLO26, MiDaS và ByteTrack được đưa vào Fusion Pipeline để tổng hợp thông tin. Tại đây, hệ thống kết hợp thông tin về loại đối tượng, vị trí, hướng di chuyển và khoảng cách nhằm phân tích mức độ nguy hiểm của từng vật cản đối với người dùng. Dựa trên kết quả phân tích, hệ thống Danger Analyzer sẽ xác định mức độ cảnh báo phù hợp. Nếu phát hiện vật cản nguy hiểm hoặc đối tượng đang di chuyển đến gần người dùng, hệ thống sẽ sinh nội dung cảnh báo tương ứng. Phương pháp sử dụng Text-to-Speech (TTS) tiếng Việt thông qua gTTS hoặc eSpeak để chuyển nội dung cảnh báo thành giọng nói [13]. Cảnh báo âm thanh được phát theo thời gian thực nhằm hỗ trợ người khiếm thị nhận biết môi trường xung quanh và nâng cao độ an toàn trong quá trình di chuyển.

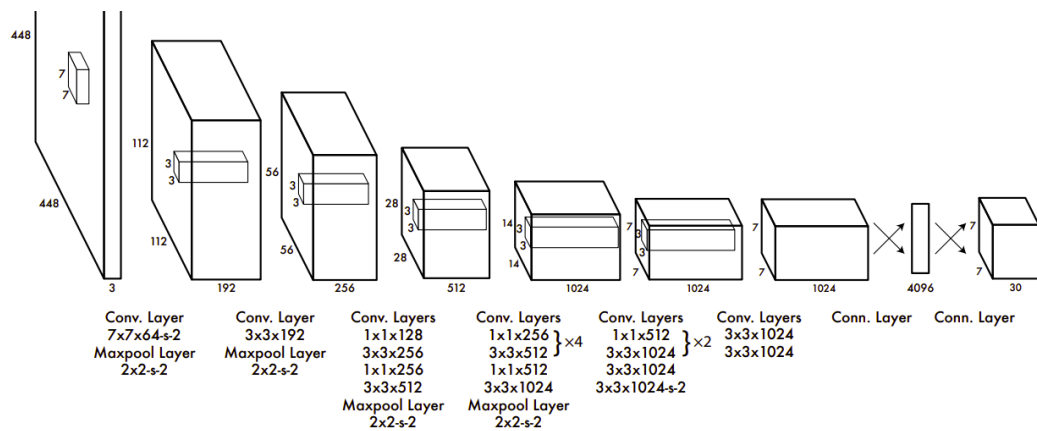
Ngoài ra, hệ thống còn được triển khai với FastAPI và WebSocket ở phía backend nhằm hỗ trợ xử lý ảnh/video và truyền dữ liệu thời gian thực. Giao diện frontend được xây dựng bằng React và Vite, giúp hiển thị trực quan kết quả nhận diện và hỗ trợ quản lý.

Chương 2 CƠ SỞ LÝ THUYẾT

2.1 Mô hình phát hiện đối tượng YOLO

YOLO là 1 từ viết tắt cho ‘You only look once’ (Bạn chỉ nhìn một lần) là một thuật toán phát hiện đối tượng chia hình ảnh thành một hệ thống lưới [14]. Mỗi ô trong lưới chịu trách nhiệm phát hiện các đối tượng trong chính nó. YOLO là một trong những thuật toán phát hiện đối tượng nổi tiếng nhất do tốc độ và độ chính xác của nó.

YOLO là một mô hình mạng CNN cho việc phát hiện, nhận dạng, phân loại đối tượng. YOLO được tạo ra từ việc kết hợp giữa các convolutional layers và connected layers [14]. Trong đó các convolutional layers sẽ trích xuất ra các feature của ảnh, còn fullconnected layers sẽ dự đoán ra xác suất đó và tọa độ của đối tượng.



Hình 2.1: Mô hình mạng CNN được sử dụng trong YOLO [15]

Hình ảnh đầu vào sẽ được chia thành một lưới kích thước $S \times S$, trong đó mỗi ô lưới chịu trách nhiệm phát hiện các đối tượng có tâm nằm trong ô đó. Đối với mỗi ô, mô hình sẽ dự đoán một số lượng bounding box cùng với thông tin về vị trí và xác suất lớp.

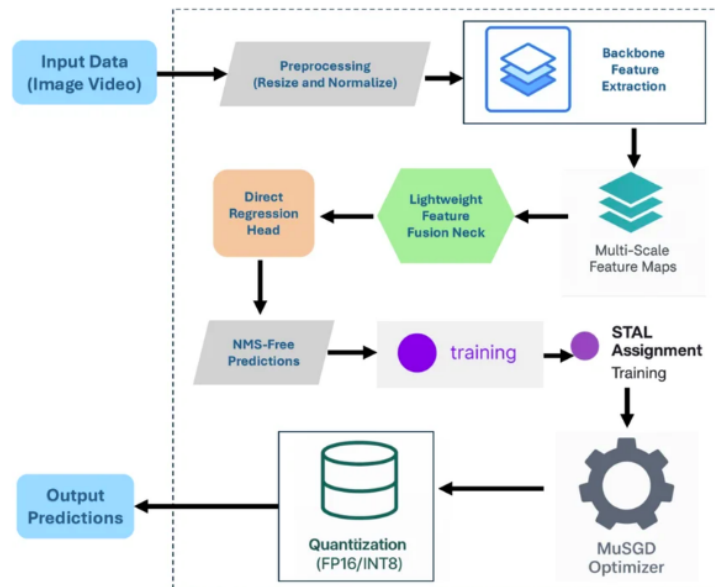
Với mỗi bounding box, mô hình dự đoán 5 tham số bao gồm $(x, y, w, h, confidence)$. Trong đó, (x, y) là tọa độ tâm của bounding box tương đối so với ô lưới, còn (w, h) là

chiều rộng và chiều cao được chuẩn hóa theo kích thước toàn ảnh. Giá trị *confidence* thể hiện mức độ tin cậy của bounding box, được tính dựa trên xác suất tồn tại đối tượng và độ chồng lấp giữa bounding box dự đoán và ground truth. Ngoài ra, mỗi ô lưới còn dự đoán xác suất thuộc về các lớp đối tượng dưới dạng $Pr(Class_i | Object)$. Xác suất cuối cùng của một lớp được tính bằng cách kết hợp giữa xác suất lớp và *confidence* của bounding box [15].

Qua nhiều thế hệ cải tiến, kiến trúc YOLO liên tục được tinh chỉnh theo hai hướng: trích xuất đặc trưng đa tỉ lệ (từ YOLOv3 với FPN đến các mô-đun C3, C2f, C3k2 ở các phiên bản sau) và phương pháp gán nhãn, từ neo cố định (*anchor-based*) sang không neo (*anchor-free*) với chiến lược gán nhãn động TAL (*Task-Aligned Learning*) từ YOLOv8 trở đi.

2.1.1 Mô hình YOLO26

YOLO26 là thế hệ mới nhất thuộc họ mô hình phát hiện đối tượng You Only Look Once (YOLO) do Ultralytics phát triển, được giới thiệu vào khoảng cuối năm 2025 [3]. Khác với các phiên bản trước chủ yếu tập trung vào việc gia tăng độ chính xác thông qua việc mở rộng kích thước mô hình, YOLO26 được thiết kế lại nhằm tối ưu hóa cho việc triển khai trên các thiết bị biên (edge devices), hệ thống nhúng và môi trường có tài nguyên hạn chế.

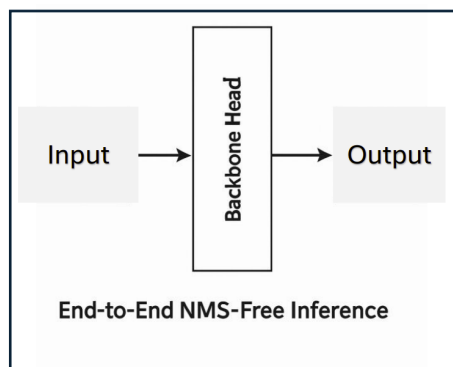


Hình 2.2: Kiến trúc tổng quan YOLO26

1. Kiến trúc NMS-Free

Các phiên bản YOLO trước đây (như YOLOv8, YOLOv9) yêu cầu bước hậu xử lý Non-Maximum Suppression (NMS) để loại bỏ các bounding box trùng lặp. Tuy nhiên, YOLO26 được thiết kế theo hướng end-to-end hoàn chỉnh, cho phép mô hình trực tiếp xuất ra các dự đoán cuối cùng mà không cần áp dụng NMS [3].

Việc loại bỏ bước NMS giúp giảm độ trễ của toàn hệ thống và đơn giản hóa pipeline suy luận, đặc biệt phù hợp với các ứng dụng thời gian thực.

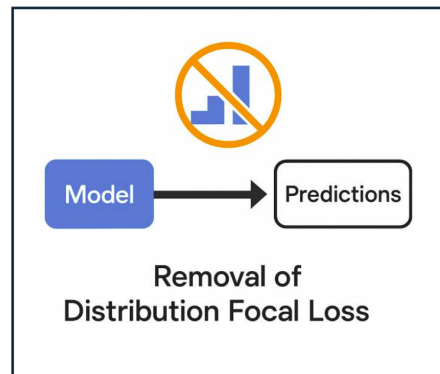


Hình 2.3: Kiến trúc NMS-Free [3]

2. Loại bỏ Distribution Focal Loss

Trong các phiên bản trước, Distribution Focal Loss (DFL) được sử dụng để cải thiện độ chính xác của bounding box. Tuy nhiên, DFL gây khó khăn trong việc chuyển đổi mô hình sang các định dạng triển khai như TensorRT, ONNX hay CoreML.

YOLO26 loại bỏ hoàn toàn DFL, giúp quá trình export và triển khai mô hình trở nên đơn giản, ổn định và tương thích tốt hơn với phần cứng [3].

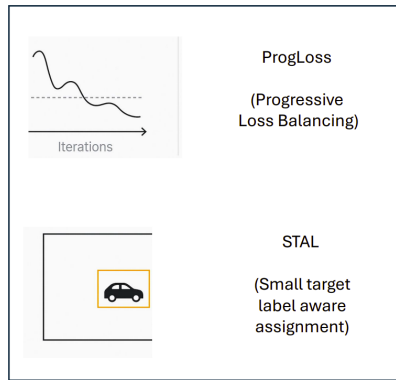


Hình 2.4: Loại bỏ DFL [3]

3. Tối ưu phát hiện vật thể nhỏ

YOLO26 áp dụng cơ chế gán nhãn mới mang tên Small-Target-Aware Label Assignment (STAL), kết hợp với hàm mất mát cải tiến. Cơ chế này giúp mô hình nhạy hơn với các vật thể nhỏ hoặc ở khoảng cách xa [3].

Nhờ đó, hiệu năng phát hiện các đối tượng nhỏ như biển số xe hoặc chi tiết chữ viết tay được cải thiện đáng kể so với các phiên bản trước.

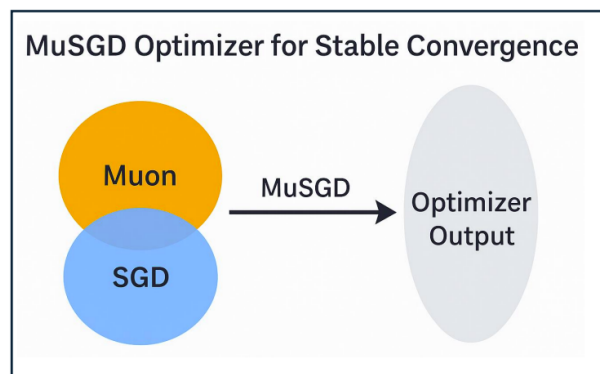


Hình 2.5: Tối ưu cho vật thể nhỏ [3]

4. Trình tối ưu hóa MuSGD

YOLO26 sử dụng trình tối ưu hóa MuSGD, kết hợp giữa Stochastic Gradient Descent (SGD) và thuật toán Muon. Phương pháp này giúp cải thiện tốc độ hội tụ và đảm bảo sự ổn định trong quá trình huấn luyện [3].

Việc áp dụng MuSGD cho phép mô hình đạt hiệu năng cao mà vẫn duy trì độ ổn định khi huấn luyện trên các tập dữ liệu lớn.



Hình 2.6: Tối ưu hóa MuSGD [3]

Nhờ đó, YOLO26 có thể được sử dụng như một nền tảng thống nhất cho nhiều ứng dụng khác nhau trong thị giác máy tính.

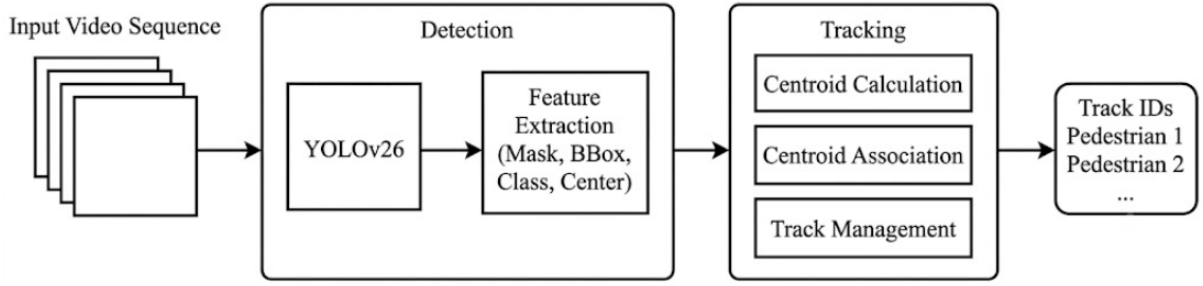
2.2 Multi-Object Tracking

Trong lĩnh vực thị giác máy tính, bài toán theo dõi đa đối tượng (*Multi-Object Tracking* - MOT) đóng vai trò cốt lõi trong các ứng dụng giám sát video, phân tích hành vi và xe tự hành [12]. Nghiên cứu này tiếp cận bài toán theo mô hình "Theo dõi thông qua phát hiện" (*Tracking-by-Detection*). Quy trình này bắt đầu bằng việc sử dụng một mạng nơ-ron chuyên biệt để phát hiện và xác định vị trí các hộp bao của đối tượng trong từng khung hình độc lập, sau đó áp dụng một thuật toán liên kết dữ liệu nhằm kết nối các hộp bao này qua chuỗi thời gian để tạo thành các quỹ đạo chuyển động nhất quán.

2.2.1 Phương pháp Centroid Tracking

Phương pháp này lựa chọn phát triển mô-đun theo dõi đối tượng dựa trên phương pháp Centroid Tracking (bám vết theo tâm hình học) [16]. Đây là một hướng tiếp cận nhẹ, tận dụng đặc tính liên tục của chuyển động trong chuỗi video, theo đó vị trí của cùng một đối tượng giữa hai khung hình liên tiếp chỉ thay đổi trong một phạm vi nhỏ khi tốc độ khung hình đủ cao.

Khác với các phương pháp theo dõi hiện đại sử dụng mạng học sâu để trích xuất đặc trưng và thực hiện liên kết dữ liệu (data association), giải pháp đề xuất loại bỏ hoàn toàn các mô hình học sâu ở tầng theo dõi nhằm giảm thiểu chi phí tính toán. Thay vào đó, quá trình gán định danh và duy trì quỹ đạo được thực hiện thông qua các phép toán hình học đơn giản trên mặt phẳng ảnh, dựa trực tiếp vào tọa độ pixel của tâm đối tượng được phát hiện từ camera.



Hình 2.7: Kiến trúc Centroid Tracking.

Nhờ cấu trúc thuật toán tinh gọn, phương pháp không chỉ đảm bảo khả năng theo dõi thời gian thực mà còn phù hợp với các nền tảng phần cứng nhưng có tài nguyên hạn chế. Luồng xử lý chi tiết cùng kiến trúc phân cấp chức năng của mô-đun theo dõi được trình bày trong Hình 2.7.

1. Detection: Chuỗi khung hình video đầu vào (*Input Video Sequence*) được xử lý bởi mạng YOLO26 để trích xuất ra thông tin vị trí hộp bao (*BBox*) và nhãn phân lớp (*Class*) của các chướng ngại vật xung quanh người khiếm thị [3]. Toàn bộ tập hợp dữ liệu này tạo thành mảng đầu vào *detections* sẵn sàng cung cấp cho module theo dõi tâm đối tượng.

2. Centroid Calculation and Association: Phương pháp tiến hành thu hẹp thông tin bằng cách tính toán tọa độ điểm tâm (*Centroid*) của từng hộp bao [16]. Tại mỗi khung hình hiện tại, một ma trận chi phí được thiết lập dựa trên khoảng cách Euclide tuyến tính giữa tập hợp các tâm đối tượng đang quản lý trong bộ nhớ và các tâm mới xuất hiện:

$$d = \sqrt{(x_{\text{object}} - x_{\text{input}})^2 + (y_{\text{object}} - y_{\text{input}})^2} \quad (2.1)$$

Thuật toán áp dụng chiến lược gán cặp láng giềng gần nhất để liên kết các đối tượng, với điều kiện khoảng cách phải nhỏ hơn ngưỡng giới hạn không gian hình học $max_distance = 120.0$ pixel nhằm triệt tiêu các liên kết sai lệch từ vùng nền nhiễu.

3. Track Management: Điều khiển vòng đời của các đối tượng dựa trên trạng thái xuất hiện. Những hộp bao mới không trùng khớp với dữ liệu cũ được khởi tạo một mã

định danh duy nhất (*Track ID*). Ngược lại, những đối tượng bị mất dấu tạm thời do vật cản che khuất sẽ được tích lũy biến đếm vắng mặt *disappeared*. Nếu vượt quá ngưỡng sinh tồn $max_disappeared = 30$ khung hình, đối tượng sẽ bị xóa hoàn toàn khỏi bộ nhớ đệm để giải phóng tài nguyên.

Việc lựa chọn kiến trúc bám vết dựa trên tâm hình học tinh gọn này mang lại một lợi thế thực tiễn quan trọng. Bằng cách loại bỏ hoàn toàn khối trích xuất đặc trưng ngoại quan công kênh hay các phép suy diễn phức tạp từ các mô hình AI khác ở tầng liên kết, phương pháp đã giảm thiểu tối đa độ trễ thuật toán cục bộ, ngăn chặn hiện tượng trễ tích lũy xuyên suốt pipeline, từ đó đảm bảo các lệnh thoát cảnh báo an toàn.

2.3 MIDAS

2.3.1 Bài toán ước lượng độ sâu

Ước lượng độ sâu từ một ảnh duy nhất (Monocular Depth Estimation) là một bài toán nền tảng nhưng chứa đựng tính nghịch đảo cao (ill-posed problem), do một hình ảnh hai chiều có thể được tạo ra từ vô số cấu trúc ba chiều khác nhau trong không gian thực [11]. Sự phát triển của các kiến trúc học sâu đã mở ra khả năng giải quyết bài toán này bằng cách huấn luyện các mạng nơ-ron hàm ý nhận diện các dấu hiệu phối cảnh, cấu trúc hình học và phân phối ánh sáng. Tuy nhiên, một rào cản lớn đối với các mô hình ước lượng độ sâu hiện đại là khả năng tổng quát hóa kém (generalization capability). Phần lớn các mạng nơ-ron khi được huấn luyện trên một tập dữ liệu cụ thể thường có xu hướng học thuộc lòng các đặc trưng phân phối và cấu hình nội tại của camera thuộc tập dữ liệu đó (chẳng hạn như bối cảnh không gian trong nhà của NYU Depth-v2 [17] hoặc bối cảnh giao thông của KITTI [18]). Khi áp dụng trực tiếp vào các môi trường mới lạ hoàn toàn mà không trải qua quá trình tinh chỉnh (Zero-shot transfer), các mô hình này thường cho ra kết quả dự đoán sai lệch nghiêm trọng.



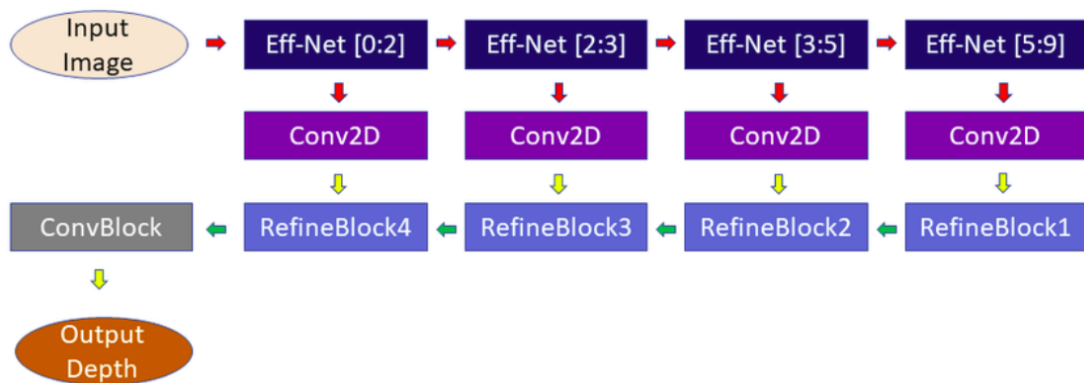
Hình 2.8: Hình ảnh minh họa [11]

Động lực cốt lõi của nghiên cứu MiDaS xuất phát từ nhận định rằng nguyên nhân chính hạn chế khả năng tổng quát hóa của mô hình không nằm ở năng lực của kiến trúc mạng, mà phụ thuộc vào quy mô và tính đa dạng của dữ liệu huấn luyện. Để xây dựng một mô hình có độ bền vững cao (robust) trong mọi điều kiện thực tế, giải pháp tối ưu là phải thực hiện huấn luyện đồng thời trên nhiều nguồn dữ liệu khác nhau. Mặc dù vậy, việc kết hợp trực tiếp các tập dữ liệu độ sâu lại đối mặt với những thách thức kỹ thuật rất lớn do sự không tương thích về mặt biểu diễn toán học.

Để giải quyết triệt để bài toán đồng hóa dữ liệu phức tạp này, thuật toán MiDaS đã đề xuất một phương pháp luận đột phá. Thay vì cố gắng ép buộc các tập dữ liệu tuân theo một quy chuẩn vật lý tuyệt đối, thiết kế một hàm mất mát mới có khả năng thích ứng linh hoạt với các biểu diễn độ sâu khác nhau thông qua việc triệt tiêu hoàn toàn các yếu tố về tỷ lệ (scale) và độ dịch chuyển (shift). Sự ra đời của MiDaS đã thiết lập một tiêu chuẩn mới cho bài toán ước lượng độ sâu đơn sắc, cho phép khai thác triệt để thế mạnh phối hợp của nhiều tập dữ liệu quy mô lớn nhằm tạo ra một mô hình có khả năng dự đoán độ sâu ổn định, chính xác trong các môi trường thực tế ngẫu nhiên mà không cần điều chỉnh lại tham số mạng.

2.3.2 Kiến trúc tổng quan của mô hình MiDaS

Kiến trúc tổng quan của MiDaS được xây dựng dựa trên nguyên lý của các mạng nơ-ron học sâu tích chập cấu trúc cao cấp, áp dụng mô hình mã hóa - giải mã (Encoder-Decoder) kết hợp với các kết nối tắt (skip connections) nhằm tối ưu hóa việc trích xuất và khôi phục đặc trưng đa quy mô [11]. Ở nhánh mã hóa, MiDaS được thiết kế để có thể linh hoạt tích hợp các mạng backbone mạnh mẽ như ResNet, ResNeXt, hoặc các cấu trúc Transformer thị giác (Vision Transformer - ViT) trong các phiên bản nâng cấp. Nhánh mã hóa đóng vai trò trích xuất các đặc trưng ngữ cảnh mang tính toàn cục (global context) và cấu trúc hình học thô của bức ảnh, giúp mạng nơ-ron hiểu được mối quan hệ không gian tổng thể giữa các thực thể và nền cảnh, bất kể hình ảnh thuộc miền dữ liệu nào.



Hình 2.9: Kiến trúc mô hình MIDAS [11]

Hình 2.9 tại nhánh giải mã, mô hình áp dụng cấu trúc mạng tinh chỉnh đa quy mô (Multi-Scale Refinement Network) dựa trên chuỗi các khối tinh chỉnh liên kết (RefineBlock) nhằm khôi phục từng bước độ phân giải không gian của bản đồ độ sâu đầu ra. Do đặc trưng trích xuất từ các tầng mạng EfficientNet (Eff-Net) mang các kích thước hình học khác nhau, chúng được đưa qua các lớp tích chập hai chiều (Conv2D) tương ứng để đồng bộ hóa số lượng kênh (channels). Quá trình này giúp chuẩn hóa dữ liệu trước khi nạp vào nhánh giải mã qua các đường kết nối tắt (skip connections) [19].

Luồng xử lý tại bộ giải mã được thực hiện theo cơ chế liên kết ngược, bắt đầu từ tầng sâu nhất là RefineBlock₁ (nhận đặc trưng từ cụm lớp Eff-Net[5 : 9]). Thông tin ngữ cảnh cao cấp từ khối phía trước sẽ được truyền ngược theo chiều ngang sang khối tiếp theo (RefineBlock₂ → RefineBlock₄), đồng thời tích hợp trực tiếp với các thông tin biên cạnh sắc nét từ tầng nông đi xuống từ các khối Conv2D thông qua phép toán cộng hợp nhất $\oplus$. Sự kết hợp đa quy mô này cho phép mạng nơ-ron liên tục bù đắp lượng thông tin không gian bị hao hụt trong quá trình mã hóa tích chập.

Ở giai đoạn cuối cùng, đặc trưng tổng hợp từ RefineBlock₄ được chuyển qua một khối tích chập kết thúc (ConvBlock) để tinh chỉnh và làm mịn bề mặt. Cơ chế này đảm bảo rằng bản đồ độ sâu dự đoán đầu ra (Output Depth) không chỉ phản ánh chính xác xu hướng phân phối khoảng cách cấu trúc ba chiều tổng thể, mà còn duy trì được độ tách biệt rõ ràng tại ranh giới của các đối tượng, giảm thiểu hiện tượng mờ nhòe hoặc lem biên tại vùng chuyển tiếp giữa tiền cảnh và hậu cảnh.

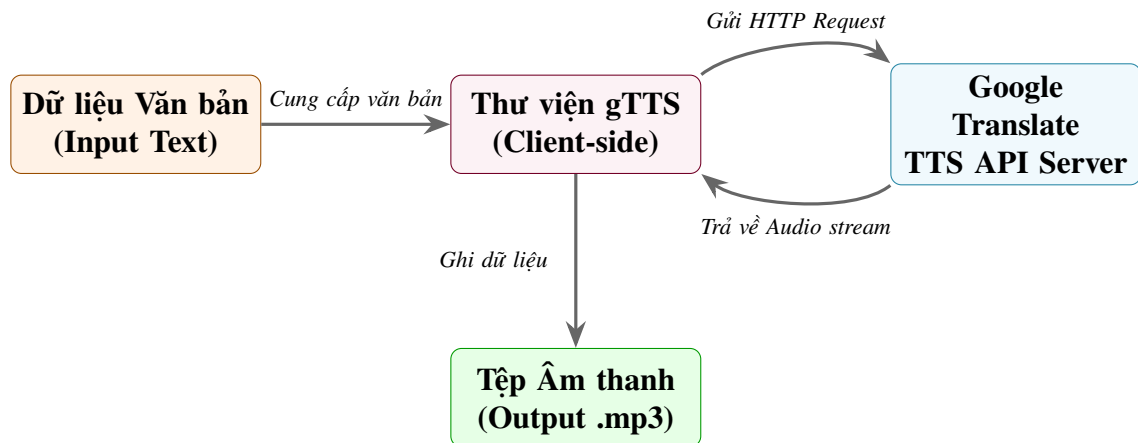
2.4 gTTS

2.4.1 *Khái niệm*

gTTS (Google Text-to-Speech) là một thư viện mã nguồn mở dành cho ngôn ngữ lập trình Python, đồng thời tích hợp công cụ giao diện dòng lệnh (CLI). Công cụ này đóng vai trò như một cầu nối trung gian, cho phép các ứng dụng giao tiếp trực tiếp với giao diện lập trình ứng dụng dịch thuật của Google (Google Translate's Text-to-Speech API). Nhiệm vụ cốt lõi của gTTS là thực hiện quá trình chuyển đổi các chuỗi văn bản viết thành dữ liệu âm thanh nói một cách tự động và thông minh. Kết quả đầu ra của quá trình xử lý này thường được trích xuất và lưu trữ dưới định dạng tệp tin âm thanh tiêu chuẩn .mp3, hoặc được truyền dẫn trực tiếp vào các luồng xử lý dữ liệu nhị phân (byte stream) nhằm phục vụ cho các ứng dụng đa phương tiện tương tác theo thời gian thực.

2.4.2 Cơ chế hoạt động

Cơ chế vận hành của thư viện gTTS tuân thủ chặt chẽ theo mô hình Client-Server dựa trên giao thức HTTP truyền thông thông qua môi trường mạng Internet. Quy trình xử lý toàn diện được diễn ra qua bốn giai đoạn tuần tự.



Hình 2.10: Sơ đồ khối cơ chế hoạt động của hệ thống gTTS

Đầu tiên là giai đoạn khởi tạo dữ liệu, trong đó ứng dụng phía người dùng (Client) sẽ chuẩn bị chuỗi văn bản cần chuyển đổi, đồng thời thiết lập các thông số cấu hình đi kèm bao gồm mã ngôn ngữ định dạng và tốc độ phát âm mong muốn.

Tiếp theo, ở giai đoạn phân đoạn và gửi yêu cầu, thư viện gTTS sẽ tự động tính toán và phân tách các đoạn văn bản dài thành các chuỗi ký tự ngắn hơn dựa trên hệ thống dấu câu thực tế. Cơ chế này giúp đảm bảo dữ liệu truyền đi không vượt quá giới hạn độ dài ký tự nghiêm ngặt do API miễn phí của Google quy định. Sau khi hoàn tất đóng gói, gTTS gửi một yêu cầu HTTP POST hoặc GET trực tiếp đến máy chủ Google Translate.

Giai đoạn thứ ba diễn ra hoàn toàn tại phía máy chủ (Server-side). Hệ thống đám mây của Google sau khi tiếp nhận yêu cầu sẽ áp dụng các thuật toán Học máy tiên tiến cùng công nghệ Xử lý ngôn ngữ tự nhiên (NLP) để phân tích sâu cú pháp văn bản, xác định trọng âm, thiết lập ngữ điệu phù hợp và tổng hợp thành một luồng âm thanh kỹ thuật số hoàn chỉnh.

Cuối cùng, tại giai đoạn nhận kết quả, máy chủ gửi phản hồi chứa luồng âm thanh đã được mã hóa về lại cho Client. Thư viện gTTS tiếp nhận luồng dữ liệu này, tiến hành giải mã và thực thi việc ghi trực tiếp vào bộ nhớ tạm hoặc xuất ra thành tệp tin âm thanh vật lý trên hệ thống.

2.4.3 Các tính năng chính của thư viện

Thư viện gTTS sở hữu nhiều tính năng vượt trội, mang lại khả năng ứng dụng cao trong các dự án phần mềm. Tính năng nổi bật nhất là việc hỗ trợ kho tàng đa ngôn ngữ vô cùng phong phú. Nhờ thừa hưởng cơ sở dữ liệu toàn cầu từ Google Translate, gTTS có khả năng chuyển đổi hơn 100 ngôn ngữ khác nhau, đồng thời hỗ trợ nhận diện và phát âm chính xác theo từng biến thể giọng địa phương đặc trưng của cùng một ngôn ngữ.

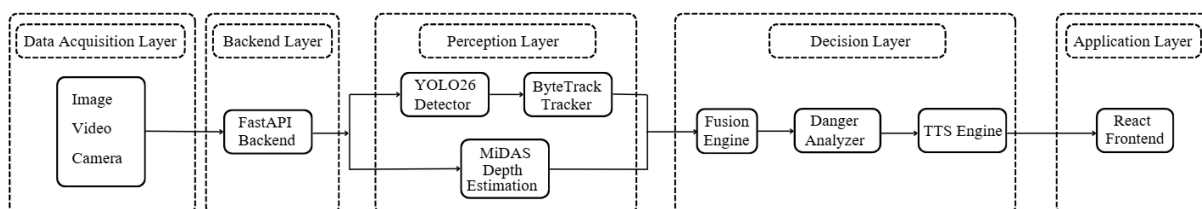
Bên cạnh đó, thư viện còn tích hợp sẵn cơ chế phân tách văn bản thông minh, tự động ngắt câu dựa trên logic ngữ pháp để đảm bảo luồng âm thanh đầu ra liên tục, không bị đứt gãy đột ngột và tối ưu hóa tối đa băng thông truyền tải. Người dùng cũng có thể linh hoạt điều chỉnh tốc độ đọc thông qua các tham số cấu hình hệ thống, cho phép hạ thấp tốc độ phát âm nhằm phục vụ tối ưu cho các ứng dụng giáo dục hoặc hỗ trợ học ngoại ngữ.

Đặc biệt, gTTS mang tính tinh gọn cao khi cho phép triển khai hoàn toàn miễn phí mà không yêu cầu người phát triển phải thiết lập tài khoản Cloud phức tạp, không cần cấu hình các khóa bảo mật API khắt khe hay chịu bất kỳ ràng buộc nào về chi phí duy trì hàng tháng.

Chương 3 PHƯƠNG PHÁP

3.1 Kiến trúc bài toán

Kiến trúc bài toán được thiết kế bao gồm năm tầng chức năng cốt lõi hoạt động theo cơ chế xử lý tuần tự. Dữ liệu được truyền qua từng tầng, trong đó mỗi tầng thực hiện một nhóm tác vụ chuyên biệt và cung cấp đầu ra cho tầng kế tiếp. Kiến trúc này bảo đảm tính phân tách chức năng rõ ràng, nâng cao khả năng tái sử dụng và mở rộng, đồng thời hỗ trợ tối ưu hóa tài nguyên tính toán.

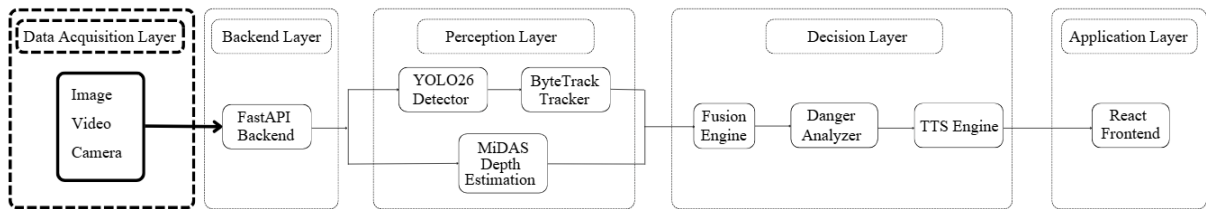


Hình 3.1: Kiến trúc tổng thể của mô hình

1. Data Acquisition Layer

Đây là tầng đầu vào (Input) của toàn bộ kiến trúc, chịu trách nhiệm tiếp nhận và tiền xử lý sơ bộ các nguồn dữ liệu thị giác. Giải pháp hỗ trợ đa dạng các nguồn đầu vào bao gồm:

- **Hình ảnh tĩnh (Image):** Các tập tin ảnh có sẵn phục vụ kiểm thử đơn lẻ.
- **Video:** Các luồng video được ghi hình trước phục vụ cho việc đánh giá độ ổn định.
- **Camera:** Luồng dữ liệu video trực tiếp từ thiết bị ngoại vi (Webcam, Camera IP) nhằm đáp ứng các ứng dụng thời gian thực.

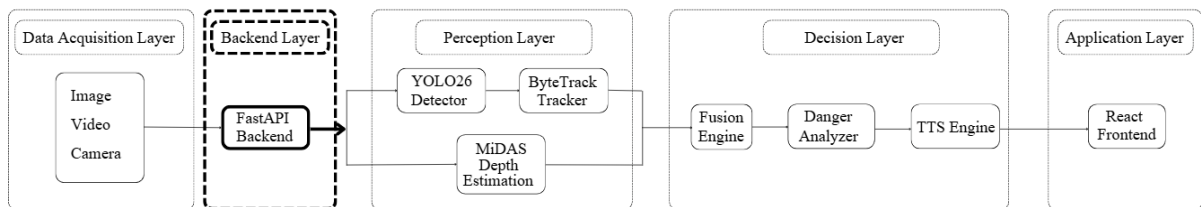


Hình 3.2: Data Acquisition Layer

2. Backend Layer

Đóng vai trò là trung tâm điều phối dữ liệu đầu vào và quản lý luồng công việc của chương trình.

- **FastAPI Backend:** Sử dụng framework FastAPI (Python) nhờ vào hiệu năng cao, hỗ trợ xử lý bất đồng bộ (Asynchronous) mạnh mẽ. Tầng này chịu trách nhiệm nhận luồng dữ liệu từ tầng thu thập, thiết lập các kết nối API, truyền dữ liệu đến tầng nhận thức một cách mượt mà và quản lý trạng thái vận hành.



Hình 3.3: Backend Layer

3. Perception Layer

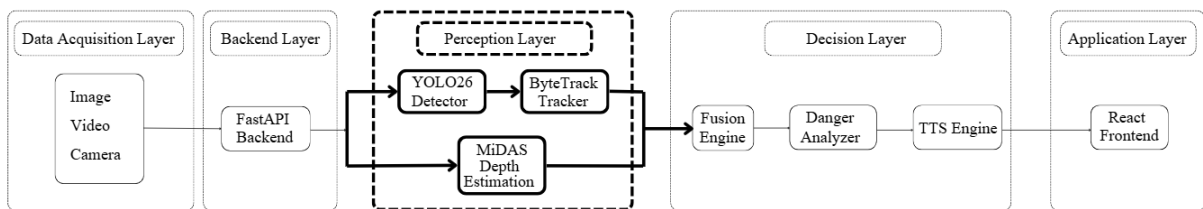
Đây là mô-đun xử lý chính của thị giác máy tính (Computer Vision), hoạt động theo mô hình xử lý song song hai nhánh để khai thác tối đa thông tin từ môi trường:

- **Nhánh phát hiện và theo dõi đối tượng:**
 - *YOLO26 Detector:* Mô hình học sâu chịu trách nhiệm phát hiện vật thể (Object Detection) trong từng khung hình với tốc độ cao và độ chính xác tối ưu [3].

- *ByteTrack Tracker*: Nhận kết quả từ YOLO26 để thực hiện gán ID và theo dõi chuyển động (Object Tracking) của các đối tượng qua từng khung hình, giúp duy trì tính liên tục của dữ liệu [12].

- **Nhánh ước tính chiều sâu:**

- *MiDAS Depth Estimation*: Hoạt động độc lập và song song với nhánh phát hiện. Mô hình này thực hiện ước tính chiều sâu đơn mục (Monocular Depth Estimation), chuyển đổi ảnh 2D thành bản đồ độ sâu (Depth Map) để xác định khoảng cách tương đối của các vật thể xung quanh [11].



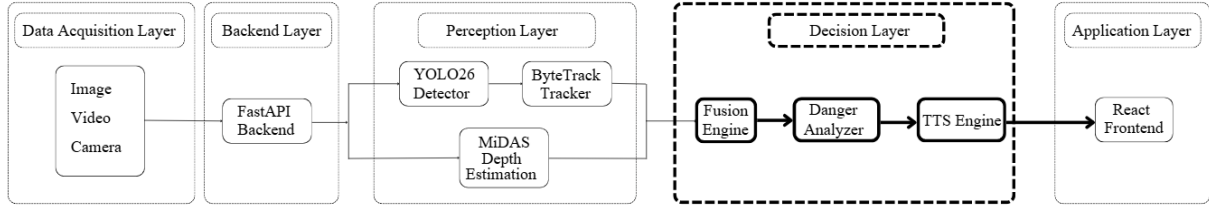
Hình 3.4: Perception Layer

4. Decision Layer

Sau khi nhận được thông tin giàu ngữ cảnh từ Perception Layer, tầng này sẽ tổng hợp và đưa ra các phản hồi thông minh:

- **Bộ dung hợp (Fusion Engine):** Tổng hợp dữ liệu từ hai nhánh (tọa độ vật thể từ ByteTrack và bản đồ độ sâu từ MiDAS) để đồng bộ hóa vị trí không gian 3D của từng đối tượng cần quan tâm.
- **Bộ phân tích nguy hiểm (Danger Analyzer):** Dựa trên dữ liệu đã dung hợp, bộ phân tích áp dụng các thuật toán hoặc quy tắc logic để đánh giá mức độ nguy hiểm (ví dụ: vật thể quá gần, có nguy cơ va chạm).

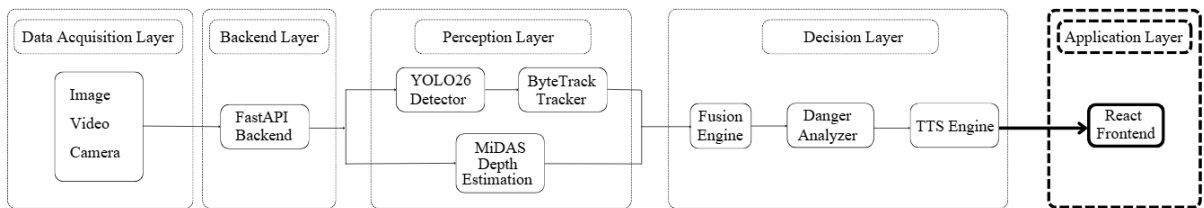
- **Công cụ chuyển văn bản thành giọng nói (TTS Engine - Text-to-Speech):**
Chuyển đổi các cảnh báo hoặc phân tích dạng văn bản thành tín hiệu âm thanh/giọng nói để chuẩn bị phản hồi cho người dùng.



Hình 3.5: Decision Layer

5. Application Layer

- **React Frontend:** Tầng giao diện người dùng (UI/UX). Được xây dựng trên thư viện React, chịu trách nhiệm nhận kết quả xử lý cuối cùng từ nền tảng hậu đài, hiển thị luồng video được trực quan hóa (vẽ khung bao bounding box, hiển thị khoảng cách) và phát ra các cảnh báo âm thanh, mang lại trải nghiệm tương tác trực quan và mượt mà cho người dùng cuối.



Hình 3.6: Application Layer

3.2 Dữ liệu thực nghiệm

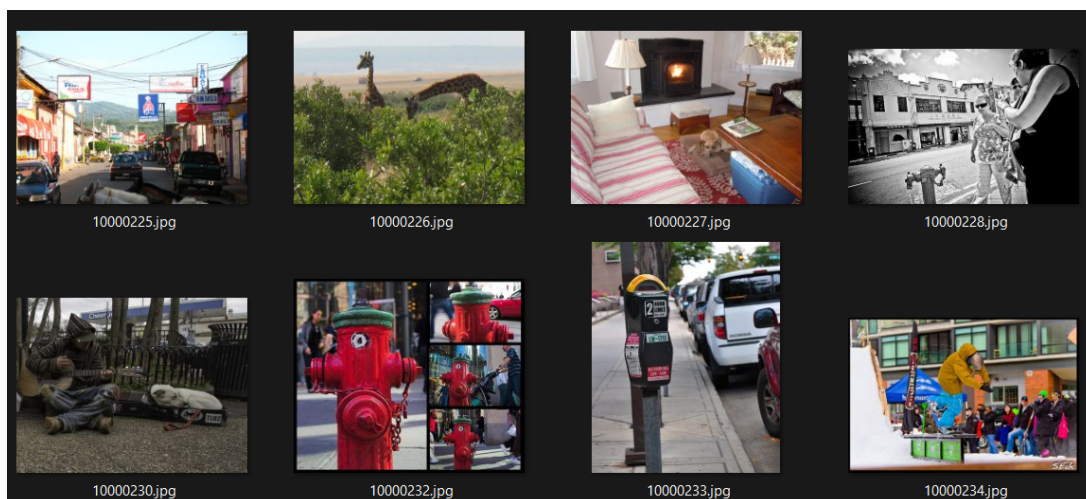
Để xây dựng tập dữ liệu phục vụ quá trình huấn luyện và đánh giá, nghiên cứu sử dụng bộ dữ liệu WOTR [20] làm nguồn dữ liệu nền tảng, đồng thời bổ sung tập dữ liệu thực tế do nhóm tự thu thập để phù hợp với ngữ cảnh ở Việt Nam. Trong khi WOTR cung cấp các mẫu dữ liệu đa dạng về đối tượng, bối cảnh và điều kiện quan sát, dữ liệu

tự xây dựng phản ánh trực tiếp môi trường hoạt động mục tiêu của hệ thống. Sự kết hợp này cho phép mô hình vừa khai thác được tính phong phú của bộ dữ liệu công khai, vừa học được các đặc trưng đặc thù của các tình huống thực tế, từ đó nâng cao độ chính xác và độ tin cậy khi triển khai.

3.2.1 Đặc điểm bộ dữ liệu

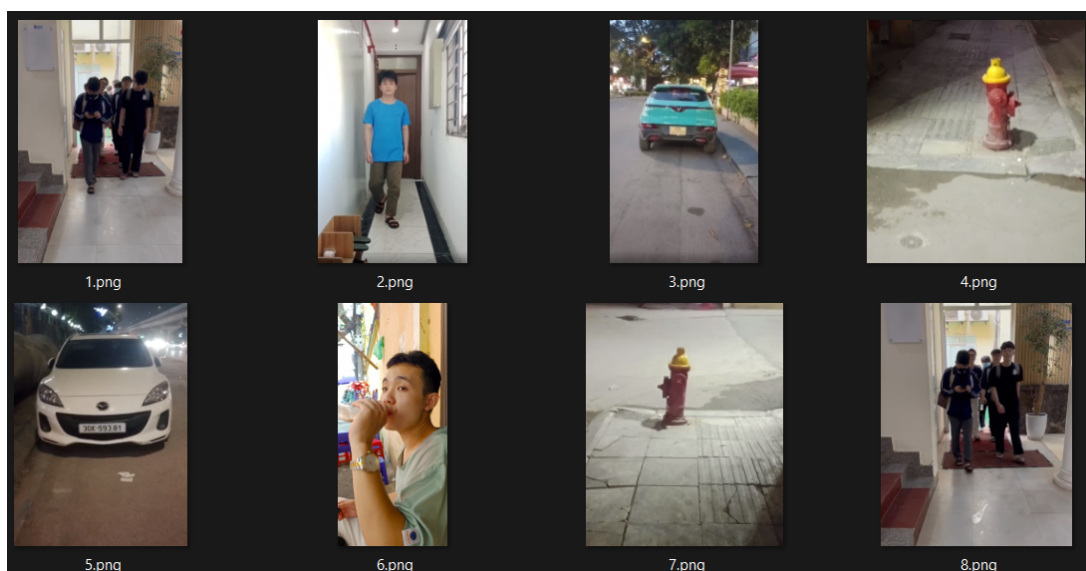
Bộ dữ liệu WOTR [20] bao gồm các tập hợp hình ảnh và chuỗi video chất lượng cao, mô phỏng góc nhìn thực tế từ các thiết bị camera hành trình và camera ngoại vi. Các đặc trưng cốt lõi của bộ dữ liệu bao gồm:

- **Sự đa dạng về đối tượng:** Chứa hàng ngàn khung hình được gán nhãn chi tiết với nhiều lớp đối tượng phổ biến trong môi trường giao thông và đời sống (người đi bộ, phương tiện giao thông, chướng ngại vật).
- **Đa dạng điều kiện môi trường:** Dữ liệu được thu thập dưới nhiều điều kiện ánh sáng khác nhau (ban ngày, ban đêm, ngược sáng) và các trạng thái thời tiết phức tạp, giúp tăng cường độ bền vững (robustness) cho các mô hình học sâu.
- **Thông tin chiều sâu tích hợp:** Bên cạnh các nhãn khung bao (bounding box) cho bài toán phát hiện, bộ dữ liệu còn cung cấp thông tin ánh xạ pixel tương ứng với khoảng cách thực tế, phục vụ trực tiếp cho việc kiểm định nhánh ước tính chiều sâu đơn mục.



Hình 3.7: Hình ảnh minh họa bộ dữ liệu WOTR [20]

Bên cạnh việc sử dụng các bộ dữ liệu công khai, nhóm nghiên cứu còn xây dựng một tập dữ liệu thực tế nhằm nâng cao khả năng thích nghi của mô hình đối với môi trường triển khai. Dữ liệu được thu thập trực tiếp bằng cách chụp ảnh tại nhiều địa điểm khác nhau, bao gồm các khu vực đường phố, khuôn viên trường học và không gian công cộng. Quá trình thu thập được thực hiện trong nhiều điều kiện khác nhau về ánh sáng, mật độ giao thông và bối cảnh môi trường nhằm bảo đảm tính đa dạng của dữ liệu.



Hình 3.8: Hình ảnh dữ liệu được thu thập

Sau khi thu thập, toàn bộ hình ảnh được rà soát và tiến hành gán nhãn thủ công cho

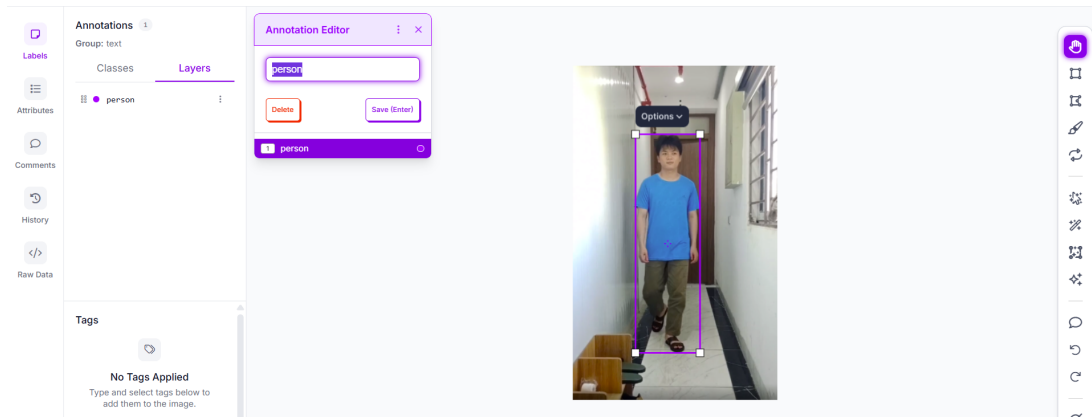
các đối tượng quan tâm. Việc đánh nhãn được thực hiện dựa trên các tiêu chuẩn thống nhất nhằm bảo đảm tính chính xác và nhất quán của dữ liệu huấn luyện. Tập dữ liệu tự xây dựng này đóng vai trò bổ sung cho các bộ dữ liệu công khai, giúp mô hình học được những đặc trưng gần với điều kiện vận hành thực tế, từ đó nâng cao hiệu quả nhận diện, theo dõi đối tượng và đánh giá mức độ nguy hiểm của hệ thống.

3.2.2 Tiền xử lý dữ liệu

Quá trình xây dựng tập dữ liệu được thực hiện trên hai nguồn dữ liệu gồm bộ dữ liệu WOTR và tập dữ liệu thực tế do nhóm tự thu thập. Đối với bộ dữ liệu WOTR [20], các ảnh đã được cung cấp kèm theo nhãn đối tượng, vì vậy nhóm chỉ tiến hành kiểm tra tính đầy đủ của dữ liệu, chuẩn hóa cấu trúc thư mục và chuyển đổi sang định dạng phù hợp với quá trình huấn luyện mô hình.

Đối với tập dữ liệu tự thu thập, nhóm tiến hành chụp ảnh tại nhiều địa điểm và trong các điều kiện môi trường khác nhau nhằm bảo đảm tính đa dạng của dữ liệu. Sau khi thu thập, các ảnh được rà soát để loại bỏ những mẫu không đạt chất lượng, chẳng hạn như ảnh bị mờ, thiếu sáng hoặc chứa quá ít thông tin hữu ích.

Tiếp theo, quá trình gán nhãn được thực hiện thủ công trên nền tảng Roboflow như Hình 3.9. Đối với mỗi ảnh, các đối tượng quan tâm được xác định và khoanh vùng bằng khung giới hạn (bounding box), đồng thời được gán vào lớp tương ứng theo hệ thống nhãn đã xây dựng. Việc gán nhãn thủ công giúp bảo đảm độ chính xác của dữ liệu, đặc biệt đối với các đối tượng xuất hiện trong những bối cảnh thực tế đặc thù mà các bộ dữ liệu công khai chưa phản ánh đầy đủ. Sau khi hoàn thành quá trình gán nhãn, dữ liệu được xuất dưới định dạng YOLO và hợp nhất với bộ dữ liệu WOTR để tạo thành tập dữ liệu huấn luyện thống nhất.



Hình 3.9: Gán nhãn dữ liệu

3.2.3 Đặc trưng dữ liệu

Để đánh giá đặc điểm của tập dữ liệu phục vụ quá trình huấn luyện và kiểm thử mô hình, nghiên cứu tiến hành phân tích sự phân bố của các lớp đối tượng trên tập dữ liệu tổng hợp, được xây dựng từ bộ dữ liệu WOTR [20] kết hợp với dữ liệu thực tế do nhóm tự thu thập và gán nhãn thủ công. Việc thống kê số lượng mẫu của từng lớp đối tượng giúp đánh giá mức độ đa dạng, tính cân bằng của dữ liệu cũng như khả năng bao quát các tình huống thực tế mà hệ thống có thể gặp phải khi triển khai. Kết quả phân bố số lượng nhãn (instances) của từng lớp đối tượng trong tập dữ liệu thông qua Bảng 3.1.

Bảng 3.1: Thống kê số lượng nhãn chi tiết theo từng lớp đối tượng.

STT	Lớp đối tượng	Số lượng nhãn	STT	Lớp đối tượng	Số lượng nhãn
1	person	28.062	11	green_light	3.981
2	pole	24.877	12	red_light	3.965
3	car	22.055	13	roadblock	3.437
4	tree	18.063	14	sign	2.654
5	motorcycle	9.712	15	truck	2.759
6	warning_column	8.323	16	reflective_cone	2.301
7	crosswalk	6.829	17	ashcan	2.282
8	bicycle	4.868	18	blind_road	1.960
9	bus	1.430	19	tricycle	1.278
10	fire_hydrant	1.120	20	dog	818

Bộ dữ liệu bao gồm các lớp đối tượng đặc trưng trong môi trường đô thị và giao thông công cộng. Trong đó, các thực thể có tần suất xuất hiện cao nhất bao gồm: *person* (người đi bộ - hơn 28.000 nhãn), *pole* (cột đèn/cột điện - gần 24.000 nhãn), *car* (ô tô - hơn 22.000 nhãn), và *tree* (cây cối - hơn 18.000 nhãn). Ngoài ra, các đối tượng hỗ trợ giao thông như *motorcycle*, *warning_column*, *bicycle*, *sign*, *green_light*, *red_light*,... cũng xuất hiện với mật độ phân tán, tạo nên một ma trận ngữ cảnh vô cùng phong phú.

3.2.4 Phân chia dữ liệu phục vụ huấn luyện

Quá trình chuẩn bị dữ liệu được thực hiện nghiêm ngặt nhằm tránh hiện tượng quá khớp (overfitting) và đảm bảo tính khách quan khi đánh giá giải pháp:

- **Tập huấn luyện (Training Set - 80%):** Được sử dụng để tối ưu hóa trọng số của mô hình YOLO26 Detector và tinh chỉnh các tham số nhận diện.

- **Tập kiểm thử (Validation Set - 20%):** Sử dụng để đánh giá liên tục trong quá trình huấn luyện, hỗ trợ điều chỉnh các siêu tham số (hyperparameters).

Bảng 3.2: Thống kê phân chia bộ dữ liệu.

Phân loại tập dữ liệu	Tỷ lệ (%)	Số lượng ảnh	Số lượng nhãn
Tập huấn luyện (Train)	80%	11142	11142
Tập kiểm thử (Validation)	20%	2786	2786
Tổng cộng	100%	13928	13928

3.3 Hàm mất mát

3.3.1 Hàm mất mát của YOLO26

Hàm mất mát cốt lõi của YOLO26 được thiết kế lại hoàn toàn cho kiến trúc End-to-End (loại bỏ hậu xử lý NMS và module DFL) [3]. Tổng hàm mất mát L_{total} là sự kết hợp có trọng số động giữa hàm phân loại (L_{cls}) và hàm hồi quy hộp bao (L_{box}):

$$L_{total}(t) = \lambda_{cls}(t) \cdot L_{cls} + \lambda_{box}(t) \cdot L_{box} \quad (3.1)$$

Trong đó, t là tiến trình huấn luyện (epoch hiện tại). Khác với các thể hệ trước, YOLO26 áp dụng cơ chế **ProgLoss (Progressive Loss Balancing)**: trọng số không cố định mà thay đổi theo thời gian. Ở giai đoạn đầu, mạng ưu tiên học phân loại đặc trưng (λ_{cls} lớn), và về cuối quá trình, hệ thống tự động dồn trọng số để tinh chỉnh ranh giới tọa độ (λ_{box} lớn) [3].

1. Hàm phân loại với Nhãn mục tiêu mềm

Để tối ưu hóa khả năng nhận diện các vật thể nhỏ (*Small-Target-Aware*), YOLO26 không sử dụng nhãn cứng (đúng/sai tuyệt đối) mà dùng nhãn mềm y_i để chấm điểm chất lượng thực tế của từng khung dự đoán:

$$L_{cls} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot (1 - \hat{p}_i)^\gamma \log(\hat{p}_i) + (1 - y_i) \cdot \hat{p}_i^\gamma \log(1 - \hat{p}_i)] \quad (3.2)$$

- N : Tổng số lượng mẫu dữ liệu.
- $\hat{p}_i$: Xác suất dự đoán của mô hình.
- y_i : Điểm số mục tiêu mềm do cơ chế **STAL (Soft Target Assignment Loss)** gán tự động ($y_i \in [0, 1]$).
- γ : Hệ số hội tụ giúp giảm nhiễu từ các mẫu nền (mẫu âm tính dễ).

2. Hàm hồi quy hộp bao trực tiếp

Bằng việc loại bỏ hoàn toàn module DFL (*Distribution Focal Loss*) nhằm giảm tải tính toán cho các thiết bị Edge CPU, việc tối ưu tọa độ trong YOLO26 phụ thuộc hoàn toàn vào **CIoU Loss (Complete IoU)**. Hàm này ép mô hình phải học đồng thời 3 yếu tố: diện tích giao nhau, khoảng cách tâm và tỷ lệ khung hình:

$$L_{box} = 1 - IoU + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v \quad (3.3)$$

- IoU : Tỷ lệ phần diện tích giao nhau trên phần hợp nhất.
- $\rho^2(\mathbf{b}, \mathbf{b}^{gt})$: Bình phương khoảng cách giữa hai tâm của hộp dự đoán và hộp chuẩn (Ground Truth).
- c : Chiều dài đường chéo của hộp đóng kín nhỏ nhất chứa cả hai hộp.
- αv : Thành phần phạt độ lệch tỷ lệ khung hình (*Aspect Ratio*), ép hộp dự đoán phải ôm sát vật thể thực tế.

3.3.2 Hàm mất mát của MiDAS

Mô hình MiDAS (Monocular Depth Estimation) chịu trách nhiệm dự đoán bản đồ độ sâu từ ảnh hai chiều (2D) tại nhánh cấu trúc song song. Do các tập dữ liệu ảnh chiều sâu thực tế được thu thập từ nhiều nguồn thiết bị (như cảm biến LiDAR, Camera Stereo, hoặc cấu trúc cấu tạo từ đồ họa máy tính) có sự sai khác lớn về dải đo, hàm mất mát thông thường như MSE hay MAE sẽ không thể hội tụ.

Hàm mất mát của MiDaS sử dụng cơ chế bất biến đối với phép co giãn tỷ lệ (scale) và dịch chuyển (shift), giúp giảm ảnh hưởng của sự khác biệt về thang đo độ sâu giữa các tập dữ liệu [11]. Hàm mất mát này được xây dựng từ hai thành phần toán học chính:

a) Hàm mất mát chiều sâu bất biến

Trước khi tính toán độ sai lệch, các bản đồ chiều sâu dự đoán d và bản đồ chiều sâu thực tế (ground-truth) d^* được đưa về một không gian chuẩn hóa chung thông qua phép biến đổi triệt tiêu sự ảnh hưởng của tỷ lệ (s) và độ dịch chuyển (t). Hàm mất mát được định nghĩa bằng công thức:

$$L_{ssi}(\hat{d}, d^*) = \frac{1}{2N} \sum_{i=1}^N |\hat{d}_i - d_i^*| \quad (3.4)$$

Trong đó, N là tổng số pixel có nhãn hợp lệ trên ảnh. Bản đồ chiều sâu chuẩn hóa $\hat{d}$ của một biểu diễn chiều sâu d được tính toán thông qua việc tối ưu hóa hai tham số s và t bằng cách giải hệ thức bình phương tối thiểu để tối thiểu hóa khoảng cách thực tế:

$$\hat{d} = \frac{d - t}{s} \quad (3.5)$$

Các tham số chuẩn hóa cấu trúc (s, t) được xác định trực tiếp theo phân phối của từng khung hình cụ thể như sau:

$$t(d) = \text{median}(d) \quad (3.6)$$

$$s(d) = \frac{1}{N} \sum_{i=1}^N |d_i - t(d)| \quad (3.7)$$

Việc sử dụng giá trị trung vị (*median*) và độ sai lệch tuyệt đối trung bình giúp hàm mất mát bền vững hơn đối với các điểm dữ liệu dị biệt (*outliers*) xuất hiện tại các ranh giới xa của ảnh.

b) Hàm mất mát độ dốc đa tỷ lệ

Để đảm bảo bản đồ chiều sâu dự đoán có độ sắc nét cao tại biên giới của các thực thể (tránh hiện tượng nhòe biên dạng vật thể), MiDAS tích hợp thêm hàm mất mát dựa trên độ dốc hình học (gradient) của ảnh trên nhiều cấp độ phân giải (*multi-scale*):

$$L_{reg}(\hat{d}, \hat{d}^*) = \frac{1}{2N} \sum_{k=1}^K \sum_{i=1}^N \left(\left| \nabla_x R_i^k \right| + \left| \nabla_y R_i^k \right| \right) \quad (3.8)$$

Trong đó:

- $R_i = \hat{d}_i - \hat{d}_i^*$ là sai số chiều sâu đã chuẩn hóa tại vị trí pixel i .
- ∇_x và ∇_y tương ứng là toán tử tính đạo hàm không gian (độ dốc) theo trục hoành và trục tung để xác định biên cạnh cấu trúc.
- k đại diện cho cấp độ phân giải trong cấu trúc kim tự tháp ảnh đa quy mô (K tầng), giúp mô hình học được cả đặc trưng biên cạnh kích thước lớn (gần) lẫn các chi tiết biên nhỏ (xa).

c) Hàm mất mát tổng hợp của cấu trúc MiDAS

Hàm mất mát cuối cùng dùng để cập nhật trọng số cho mạng nơ-ron tích chập của nhánh ước tính chiều sâu là sự kết hợp tuyến tính của hai thành phần trên theo tỷ lệ trọng số α :

$$Loss_{MiDAS} = L_{ssi}(\hat{d}, \hat{d}^*) + \alpha L_{reg}(\hat{d}, \hat{d}^*) \quad (3.9)$$

Trong các cấu hình thực nghiệm tiêu chuẩn tham số α thường được thiết lập bằng 0.5. Sự kết hợp này ép mô hình vừa phải tối ưu khoảng cách chiều sâu tương đối toàn cục, vừa giữ được cấu trúc hình học 3D sắc nét của vật thể, tạo điều kiện thuận lợi cho bộ dung hợp dữ liệu (*Fusion Engine*) tính toán không gian chính xác.

3.4 Huấn luyện mô hình

Quá trình huấn luyện mô hình được thực hiện trên nền tảng điện toán đám mây Google Colab sử dụng bộ gia tốc đồ họa phần cứng GPU Tesla T4 (14.913 MiB VRAM) và thư viện Ultralytics YOLO. Tập dữ liệu WOTR được sử dụng để huấn luyện mô hình phát hiện 20 lớp đối tượng và thành phần hạ tầng giao thông thường gặp trong môi trường đô thị, bao gồm người đi bộ, phương tiện giao thông, đèn tín hiệu, vạch qua đường, chướng ngại vật và các vật thể hỗ trợ định hướng.

Trong giai đoạn tiền xử lý, toàn bộ ảnh đầu vào được tự động thay đổi kích thước về 640×640 pixel theo cơ chế mặc định của cấu trúc YOLO nhằm đảm bảo tính đồng nhất dữ liệu và tối ưu hiệu năng huấn luyện. Các phép tăng cường dữ liệu (*Data Augmentation*) tích hợp sẵn của Ultralytics YOLO được áp dụng đầy đủ nhằm cải thiện khả năng tổng quát hóa của mô hình trước các điều kiện môi trường phức tạp khác nhau. Các siêu tham số cấu hình huấn luyện chi tiết được tổng hợp cụ thể tại Bảng 3.3.

Bảng 3.3: Cấu hình thông số huấn luyện mô hình YOLO26.

Siêu tham số	Giá trị cấu hình thực nghiệm
Kiến trúc mô hình	YOLO26
Phần cứng huấn luyện	GPU Tesla T4 15GB
Số lớp đối tượng	20
Kích thước ảnh đầu vào	640×640 pixel
Epochs	30
Bộ tối ưu hóa	MuSGD
Hàm mất mát	Box Loss + Classification Loss
Kích thước Batch size	16
Chiến lược đánh giá	Sau mỗi chu kỳ (<i>per epoch</i>)
Chỉ số lựa chọn mô hình	mAP@0.5:0.95

Mô hình được huấn luyện dựa trên bài toán phát hiện đối tượng (*Object Detection*). Trong quá trình tối ưu hóa trọng số mạng, hàm mất mát tổng hợp được cấu thành từ hai thành phần toán học chính:

- **Box Loss ($Loss_{box}$):** Đánh giá sai số hình học (tọa độ, khoảng cách) giữa hộp giới hạn dự đoán (*bounding box*) và hộp giới hạn thực tế (*ground truth*).
- **Classification Loss ($Loss_{cls}$):** Đánh giá khả năng phân loại chính xác các thực thể vào đúng lớp đối tượng mục tiêu.

Hiệu năng nhận thức của mô hình được đánh giá độc lập trên tập kiểm thử thông qua các chỉ số đo lường tiêu chuẩn bao gồm:

- **Precision (P):** Tỷ lệ đối tượng dự đoán chính xác trên tổng số các dự đoán mà mô hình đưa ra.
- **Recall (R):** Tỷ lệ các đối tượng thực tế ngoài môi trường được mô hình phát hiện thành công.
- **mAP@0.5:** Chỉ số trung bình độ chính xác (*mean Average Precision*) trên tất cả các lớp đối tượng xét tại ngưỡng giao cắt IoU (*Intersection over Union*) bằng 0.5.

- **mAP@0.5:0.95:** Chỉ số đánh giá tổng quát và nghiêm ngặt nhất khi tính toán trung bình mAP trên dải ngưỡng IoU lũy tiến từ 0.5 đến 0.95 với bước nhảy 0.05.

3.5 Nguyên lý hoạt động của các phương pháp xử lý hỗ trợ điều hướng

3.5.1 Phương pháp theo dõi đa đối tượng ByteTrack

1. Phương pháp theo dõi đối tượng dựa trên đặc trưng động

Trong các kịch bản giao thông phức tạp, việc chỉ phát hiện đối tượng tĩnh tại từng khung hình độc lập thông qua mô hình YOLO26 là chưa đủ để phân tích hành vi nguy hiểm. Do đó, hệ thống tích hợp thêm mô hình theo dõi đối tượng tuyến tính (*SimpleTracker*) nhằm gán nhãn định danh nhất quán (*Persistent ID*), đồng thời ước tính vận tốc và hướng chuyển động của các thực thể xung quanh người khiếm thị.

2. Cơ chế liên kết thực thể dựa trên khoảng cách Euclide

Tại mỗi khung hình thứ k , mô hình phát hiện YOLO26 trả về một tập hợp các khung bao dự đoán. Bộ theo dõi tiến hành trích xuất tọa độ tâm hình học (*Centroid*) của từng đối tượng:

$$C_j = (x_j, y_j) = \left(x_{\min} + \frac{w}{2}, y_{\min} + \frac{h}{2} \right) \quad (3.10)$$

Trong đó:

- C_j : Tọa độ không gian hai chiều tâm của khung bao thứ j .
- x_j, y_j : Tọa độ pixel tâm hình học theo trục ngang và trục dọc của ảnh.
- $x_{\min}, y_{\min}$: Tọa độ pixel góc trên bên trái của hộp giới hạn đối tượng.
- w, h : Chiều rộng (*width*) và chiều cao (*height*) tương ứng của hộp giới hạn.

Để liên kết các tâm mới phát hiện C_{inp} với danh sách các đối tượng đang được quản lý từ khung hình trước C_{obj} , hệ thống áp dụng thuật toán so khớp lân cận gần nhất (*Nearest Neighbor Matching*) dựa trên ma trận khoảng cách Euclide không gian 2D:

$$d(C_{\text{obj}}, C_{\text{inp}}) = \sqrt{(x_{\text{obj}} - x_{\text{inp}})^2 + (y_{\text{obj}} - y_{\text{inp}})^2} \quad (3.11)$$

Trong đó:

- $d(C_{\text{obj}}, C_{\text{inp}})$: Khoảng cách không gian 2D tính bằng đơn vị pixel giữa tâm vết cũ và tâm phát hiện mới.
- $x_{\text{obj}}, y_{\text{obj}}$: Tọa độ tâm của vết đối tượng đã được đăng ký và lưu vết trong bộ đệm lịch sử.
- $x_{\text{inp}}, y_{\text{inp}}$: Tọa độ tâm của đối tượng mới được trích xuất tại khung hình hiện tại.

Quá trình tối ưu hóa liên kết cặp được thực hiện lặp qua từng thực thể, điều kiện để một đối tượng được xác định là trùng khớp với vết cũ là khoảng cách d phải nhỏ hơn ngưỡng giới hạn thực nghiệm ($\text{max_distance} = 120,0$ pixel):

$$d(C_{\text{obj}}, C_{\text{inp}}) < d_{\text{max}} \quad (3.12)$$

Trong đó, d_{max} đại diện cho bán kính quét giới hạn tối đa cho phép liên kết vết giữa hai khung hình liên tiếp để tránh hiện tượng gán nhầm định danh cho các thực thể khác nhau nằm liền kề.

3. Cơ chế quản lý vòng đời đối tượng (Register and Deregister)

Để đối phó với hiện tượng vật thể bị che khuất tạm thời hoặc mô hình nhận diện bị trượt khung hình (*false negatives*), bộ theo dõi thiết lập một cơ chế quản lý trạng thái động dựa trên biến đếm mất dấu (*disappeared*):

- **Đăng ký mới (Register):** Các tâm dữ liệu đầu vào không trùng khớp với bất kỳ vết theo dõi sẵn có nào sẽ được coi là thực thể mới xuất hiện, hệ thống tự động tăng biến chỉ mục $ID_{next} = ID_{next} + 1$ và khởi tạo bộ đệm lịch sử hành trình.
- **Duy trì và Xóa bỏ (Deregister)** Tại mỗi khung hình, nếu một định danh ID không tìm thấy thực thể liên kết, biến số khung hình mất dấu của ID đó sẽ tăng thêm 1 ($t_{miss} = t_{miss} + 1$). Nếu vượt quá ngưỡng chịu tải ($max_disappeared = 30$ khung hình, tương đương 1 giây trên luồng video), thực thể này sẽ bị xóa hoàn toàn khỏi bộ nhớ quản lý nhằm giải phóng tài nguyên.

4. Mô hình phân tích trạng thái động và xu hướng chuyển động

Sau khi đồng bộ hóa định danh thành công, bộ theo dõi trích xuất một cửa sổ lịch sử gồm 6 vị trí gần nhất ($pts = [C_{k-5}, C_{k-4}, \dots, C_k]$) nhằm tính toán vectơ dịch chuyển trung bình trên hai trục tọa độ:

$$\Delta x = \frac{1}{M-1} \sum_{i=1}^{M-1} (x_{i+1} - x_i), \quad \Delta y = \frac{1}{M-1} \sum_{i=1}^{M-1} (y_{i+1} - y_i) \quad (3.13)$$

Trong đó:

- $\Delta x, \Delta y$: Độ dịch chuyển pixel trung bình theo thời gian của đối tượng trên trục ngang và trục dọc.
- M : Kích thước độ dài của cửa sổ trích xuất chuỗi điểm hành trình cục bộ ($M = 6$).

- x_i, y_i : Tọa độ không gian của tâm đối tượng tại vị trí thứ i nằm trong chuỗi đệm hành trình.

Từ các giá trị vi phân không gian này, hệ thống rút ra hai thông tin ngữ cảnh quan trọng phục vụ phân tích:

a) Hướng chuyển động tương đối (*Direction*): Do đặc trưng phối cảnh góc nhìn của camera đeo trên người, sự thay đổi của tọa độ dọc (Δy) phản ánh trực tiếp xu hướng khoảng cách. Quy luật logic được định nghĩa:

$$\text{Direction} = \begin{cases} \text{Approaching (Đang tiến lại gần)}, & \text{nếu } \Delta y > 3 \\ \text{Receding (Đang đi ra xa)}, & \text{nếu } \Delta y < -3 \\ \text{Stationary (Đứng yên)}, & \text{nếu } -3 \leq \Delta y \leq 3 \end{cases} \quad (3.14)$$

b) Độ lớn vận tốc và Phân cấp nguy hiểm (*Speed Estimation*): Vận tốc dịch chuyển pixel tổng hợp (v) được tính bằng mô-đun toán học:

$$v = \sqrt{(\Delta x)^2 + (\Delta y)^2} \quad (3.15)$$

Trong đó, v biểu thị độ lớn vận tốc vector dịch chuyển tổng hợp của đối tượng trên mặt phẳng ảnh hai chiều. Giá trị này được đưa qua hàm ánh xạ tuyến tính chặn trên để chuẩn hóa về dải điểm số nguy hiểm $[0.1, 1.0]$ tương ứng với các phân cấp: *đứng yên* ($v < 3$), *di chuyển chậm* ($v < 10$), *di chuyển vừa* ($v < 25$) và *di chuyển nhanh* ($v \geq 25$).

Mô hình toán học thu gọn này giúp chuyển đổi các tọa độ pixel thuần túy thành các thông tin ngữ cảnh động có ý nghĩa ngữ nghĩa, làm tiền đề trực tiếp cho hệ thống phân tích nguy hiểm ra quyết định phát âm thanh cảnh báo chính xác cho người khiếm thị.

3.5.2 Phương pháp ước tính chiều sâu và phân vùng khoảng cách

Để cung cấp thông tin khoảng cách vật lý của các chướng ngại vật phía trước người khiếm thị mà không cần sử dụng hệ thống camera dòng kép (*Stereo Camera*) phức tạp, hệ thống tích hợp module ước tính chiều sâu đơn mục dựa trên kiến trúc mạng học sâu MiDAS, kết hợp giải pháp dự phòng hình học không gian [11].

1. Cơ chế nội suy và chuẩn hóa bản đồ chiều sâu MiDAS

Mô hình mạng *MiDAS_small* nhận đầu vào là khung hình đã tiền xử lý và trả về ma trận dự đoán độ phân giải thấp [11]. Để đồng bộ ma trận này về đúng kích thước gốc của ảnh nguồn phục vụ cho việc cắt vùng chướng ngại vật, hệ thống áp dụng thuật toán nội suy đa khối (*Bicubic Interpolation*) dạng liên tục:

$$D_{\text{raw}} = \text{Interpolate}(\text{MiDAS}(I), \text{size} = (H, W), \text{mode} = \text{'bicubic'}) \quad (3.16)$$

Trong đó:

- D_{raw} : Ma trận bản đồ chiều sâu thô (*raw depth map*) sau khi khôi phục độ phân giải chuẩn.
- I : Khung hình ảnh gốc đầu vào từ luồng camera.
- $\text{MiDAS}(I)$: Bản đồ chiều sâu độ phân giải thấp do mạng nơ-ron sinh ra.
- H, W : Chiều cao (*Height*) và chiều rộng (*Width*) nguyên bản của khung hình nguồn.

**Lưu ý về mặt thuật toán*: Bản chất của phép nội suy đa khối (*Bicubic Interpolation*) là mở rộng kích thước ảnh bằng cách tính toán giá trị pixel mới dựa trên trung bình trọng số của 16 pixel lân cận (4×4) xung quanh điểm ảnh đích. Phép toán này sử dụng các đa thức bậc ba để làm mịn bề mặt chuyển tiếp hình học. So với các kỹ thuật nội suy tuyến

tính (*Bilinear*) hay láng giềng gần nhất (*Nearest Neighbor*), phép nội suy đa khối triệt tiêu được hiện tượng răng cưa và mờ nhòe biên cạnh, giúp bảo toàn tính liên tục và độ mịn toán học của các đường biên phân tách chiều sâu vật cản.

Do đầu ra nguyên bản của MiDAS tỷ lệ thuận với thị sai (*Disparity*) — tức các giá trị càng lớn đại diện cho vật thể càng ở gần camera, hệ thống thực hiện chuẩn hóa tuyến tính ma trận đầu ra về dải giá trị xác suất phân phối $[0, 1]$:

$$D(y, x) = \frac{D_{\text{raw}}(y, x) - \min(D_{\text{raw}})}{\max(D_{\text{raw}}) - \min(D_{\text{raw}})} \quad (3.17)$$

Trong đó:

- $D(y, x)$: Giá trị chiều sâu đã chuẩn hóa tại tọa độ không gian (y, x) , nằm trong khoảng $[0, 1]$.
- $\min(D_{\text{raw}}), \max(D_{\text{raw}})$: Giá trị pixel nhỏ nhất và lớn nhất tìm được trên toàn bộ ma trận chiều sâu thô.

Giá trị $D(y, x) = 1$ biểu thị khoảng cách cực cận (nguy hiểm cao) và $D(y, x) = 0$ biểu thị khoảng cách vô cực (an toàn). Trong trường hợp phần cứng gặp lỗi hoặc thiếu thư viện *PyTorch*, phương pháp tự động kích hoạt cơ chế dự phòng gradient tuyến tính (*Gradient Fallback*) theo trục dọc ảnh:

$$D_{\text{fallback}}(y, x) = \frac{y}{H - 1} \quad \forall x \in [0, W - 1] \quad (3.18)$$

Trong đó, $D_{\text{fallback}}(y, x)$ là giá trị chiều sâu giả định tại tọa độ (y, x) , phân bố tăng dần từ đỉnh ảnh ($y = 0$, khoảng cách xa) xuống đáy ảnh ($y = H - 1$, khoảng cách gần) dựa theo phối cảnh mặt đất tiêu chuẩn.

2. Thuật toán trích xuất chiều sâu vùng mục tiêu

Từ khung bao vị trí của vật thể do YOLO26 cung cấp, $B = [x_1, y_1, x_2, y_2]$, hệ thống tiến hành cắt lát ma trận chiều sâu cục bộ tương ứng với vùng không gian của thực thể:

$$R = D[y_1 : y_2, x_1 : x_2] \quad (3.19)$$

Trong đó:

- R : Ma trận con (*sub-matrix*) đại diện cho bản đồ chiều sâu thu hẹp chứa riêng thực thể mục tiêu.
- x_1, y_1 : Tọa độ điểm góc trên bên trái của khung bao đối tượng mục tiêu.
- x_2, y_2 : Tọa độ điểm góc dưới bên phải của khung bao đối tượng mục tiêu.

Thay vì lấy giá trị trung bình (*Mean*) dễ bị nhiễu do nền ảnh lọt vào khung bao, hoặc lấy giá trị lớn nhất (*Max*) dễ bị sai lệch do nhiễu pixel đơn lẻ, hệ thống áp dụng hàm thống kê phân vị thứ 80 (*80th Percentile*) để tính toán điểm số khoảng cách đại diện (S):

$$S = \text{Percentile}(R, 80) \quad (3.20)$$

Trong đó, S là điểm số chiều sâu đại diện duy nhất cho toàn bộ vùng vật cản được trích xuất. Giá trị phân vị này đảm bảo điểm số S phản ánh chính xác bề mặt phân biên gần nhất của vật thể đang hướng về phía người khiếm thị. Điểm số S sau đó được ánh xạ trực tiếp sang các phân vùng nguy hiểm ngữ nghĩa dựa trên hàm điều kiện ngưỡng cố định:

$$\text{Zone} = \begin{cases} \text{Very Close (Rất gần)}, & \text{nếu } S > 0,80 \\ \text{Close (Gần)}, & \text{nếu } 0,60 < S \leq 0,80 \\ \text{Medium (Trung bình)}, & \text{nếu } 0,40 < S \leq 0,60 \\ \text{Far (Xa)}, & \text{nếu } 0,20 < S \leq 0,40 \\ \text{Very Far (Rất xa)}, & \text{nếu } S \leq 0,20 \end{cases} \quad (3.21)$$

3. Cơ chế ước tính khoảng cách hình học dự phòng

Trong các kịch bản môi trường quá tối hoặc kiến trúc phần cứng giới hạn năng lực tính toán của mạng nơ-ron, hệ thống chuyển đổi sang mô hình ước tính khoảng cách hình học dựa trên tỷ lệ diện tích khung bao (*Bounding Box Area Ratio*). Quy luật vật lý giả định rằng vật thể có diện tích chiếm quyền sở hữu khung hình càng lớn thì khoảng cách thực tế đến camera càng gần:

$$\text{Ratio} = \frac{(x_2 - x_1) \times (y_2 - y_1)}{W \times H} \quad (3.22)$$

Trong đó, Ratio là tỷ số diện tích bề mặt của hộp bao đối tượng so với tổng diện tích không gian hiển thị của một khung hình ảnh đầy đủ.

Hàm ánh xạ phi tuyến từ tỷ lệ diện tích sang điểm số chiều sâu dự phòng (S_{geom}) được thiết lập dạng hàm từng khúc theo cấu hình thực nghiệm:

$$S_{\text{geom}} = \begin{cases} 0,90, & \text{nếu } \text{Ratio} > 0,25 \implies \text{Very Close} \\ 0,70, & \text{nếu } 0,12 < \text{Ratio} \leq 0,25 \implies \text{Close} \\ 0,50, & \text{nếu } 0,05 < \text{Ratio} \leq 0,12 \implies \text{Medium} \\ 0,25, & \text{nếu } 0,01 < \text{Ratio} \leq 0,05 \implies \text{Far} \\ 0,10, & \text{nếu } \text{Ratio} \leq 0,01 \implies \text{Very Far} \end{cases} \quad (3.23)$$

Trong đó, S_{geom} đại diện cho điểm số chiều sâu tính toán bằng hình học tĩnh. Sự kết hợp song song giữa bản đồ chiều sâu học sâu MiDAS và mô hình hình học dự phòng giúp hệ thống duy trì khả năng nhận biết cự ly vật cản liên tục, đảm bảo tính an toàn tối đa cho người khiếm thị trong mọi tình huống vận hành.

3.5.3 Phương pháp tổng hợp ngôn ngữ và cơ chế dự phòng đa tầng

Để chuyển đổi các kết quả phân tích ngữ cảnh số liệu (từ YOLO26, ByteTrack và MiDAS) thành thông báo tiếng nói trực quan cho người khiếm thị, hệ thống triển khai một công cụ tổng hợp ngôn ngữ đa nền tảng (*TTSEngine*). Công cụ này tích hợp cơ chế đồng bộ hóa mã băm, sinh sẵn dữ liệu khởi động và tự động chuyển đổi dự phòng ngoại tuyến nhằm triệt tiêu độ trễ mạng và đảm bảo an toàn tối đa cho người dùng trong mọi điều kiện môi trường.

1. Cơ chế băm MD5 và quản lý bộ nhớ đệm cục bộ

Để tối ưu hóa thời gian phản hồi thời gian thực, phương pháp ngăn chặn việc lặp lại các chu kỳ yêu cầu API trực tuyến bằng cách xây dựng một kiến trúc bộ đệm lưu trữ tệp âm thanh (*Audio Disk Caching*). Khi nhận chuỗi văn bản cảnh báo T , hệ thống thực hiện mã hóa chuỗi bằng thuật toán ký số MD5 (*Message-Digest Algorithm 5*) để sinh ra một chuỗi định danh duy nhất dài 14 ký tự:

$$K = \text{SubStr}\left(\text{MD5}\left(\text{Encode}(T)\right), 0, 14\right) \quad (3.24)$$

Tên tệp tin âm thanh tương ứng được định nghĩa cố định trong không gian lưu trữ:

$$\text{Filename} = \text{"tts_"} + K + \text{".mp3"} \quad (3.25)$$

Trong đó:

- T : Chuỗi văn bản tiếng Việt cần tổng hợp tiếng nói (Ví dụ: "Chú ý! Phát hiện vật thể phía trước.").
- MD5: Hàm toán học băm chuỗi văn bản thành chuỗi thập lục phân 128-bit.
- K : Khóa băm rút gọn (14 ký tự đầu) đại diện duy nhất cho nội dung văn bản nhằm tối ưu hóa tốc độ tìm kiếm tệp tin.

2. Thuật toán sinh sẵn cảnh báo phân cấp nguy hiểm

Nhằm loại bỏ hoàn toàn độ trễ khởi tạo trong quá trình di chuyển, hệ thống áp dụng phương pháp sinh sẵn dữ liệu (*Data Pregeneration*) ngay tại thời điểm khởi động (*Startup*).

$$f(L) = \begin{cases} \text{"Đường đi an toàn. Không có nguy hiểm."}, & \text{nếu } L = \text{LOW} \\ \text{"Chú ý! Phát hiện vật thể phía trước."}, & \text{nếu } L = \text{MEDIUM} \\ \text{"Cảnh báo nguy hiểm! Có phương tiện đang tiến gần."}, & \text{nếu } L = \text{HIGH} \\ \text{"Khẩn cấp! Nguy hiểm nghiêm trọng. Dừng ngay!"}, & \text{nếu } L = \text{CRITICAL} \end{cases} \quad (3.26)$$

Toàn bộ các chuỗi văn bản $f(L)$ này được chuyển qua hàm sinh âm thanh mẫu và lưu sẵn vào ổ đĩa trước khi luồng camera hoạt động. Khi mô hình phân tích đưa ra mức độ cảnh báo, hệ thống chỉ cần trở trực tiếp đến tệp tin cấu hình sẵn, đưa độ trễ xử lý âm thanh về mức tuyệt đối $\Delta t_{\text{synth}} = 0$ ms.

3. Kiến trúc dự phòng đa tầng gTTS và eSpeak-ng Ngoại tuyến

Đối với các câu thoại biến đổi linh hoạt (không nằm trong tập sinh sẵn), hệ thống thiết lập một quy trình xử lý dự phòng hai giai đoạn liên tục (*Multi-backend Fallback Architecture*) nhằm đối phó với sự cố mất kết nối mạng Internet toàn phần:

1. **Tầng ưu tiên 1 (Trực tuyến - gTTS Backend):** Hệ thống gửi chuỗi T lên máy chủ điện toán đám mây Google thông qua thư viện *gTTS* [13]. Chất lượng âm thanh tự nhiên của mô hình sóng âm WaveNet được ưu tiên để người khiếm thị dễ nghe nhất. Điều kiện chấp nhận tệp âm thanh là dung lượng tệp tải về phải đạt tiêu chuẩn kích thước thực tế:

$$\text{Size}_{\text{gTTS}} > 1000 \text{ Bytes} \quad (3.27)$$

2. **Tầng ưu tiên 2 (Ngoại tuyến - eSpeak-ng/pyttsx3 Backend):** Nếu kết nối mạng bị gián đoạn (gTTS trả về lỗi ngoại lệ hoặc tệp tin rỗng), hệ thống ngay lập tức kích hoạt bộ tổng hợp cục bộ ngoại tuyến *pyttsx3* kết hợp bộ máy ngôn ngữ *eSpeak-ng* [13]. Hệ thống cấu hình tần số giọng nói tiếng Việt chuẩn hóa (Rate = 145 từ/phút) và mức âm lượng tối đa (Volume = 1.0). Tiến trình sinh tệp âm thanh dạng sóng được biểu diễn:

$$T \xrightarrow{\text{pyttsx3}} \text{Audio}_{\text{WAV}} \xrightarrow{\text{FFmpeg}} \text{Audio}_{\text{MP3}} \quad (3.28)$$

Tệp dữ liệu âm thanh dạng sóng (.wav) sinh ra từ lõi máy được chuyển đổi tự động sang định dạng nén (.mp3) thông qua bộ công cụ dòng lệnh *FFmpeg* với tần số lấy mẫu chuẩn hóa 22050 Hz và luồng đơn kênh (*Mono Channel*) để đồng bộ tuyệt đối với hệ

thông phát. Sự liên kết chặt chẽ giữa hai tầng trực tuyến và ngoại tuyến này đảm bảo thiết bị hỗ trợ người khiếm thị hoạt động bền bỉ, không bị ngắt quãng tín hiệu cảnh báo ngay cả khi đi vào vùng mất sóng hoặc các khu vực hầm sâu.

Chương 4 THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Kết quả thực nghiệm

Trong phần này, báo cáo tiến hành trình bày các thiết lập thực nghiệm và đánh giá hiệu quả của hệ thống đề xuất đối với bài toán nhận biết vật cản và cảnh báo an toàn hỗ trợ người khiếm thị di chuyển. Đáng chú ý, để tối ưu hóa hiệu năng và tài nguyên tính toán, hệ thống được cấu hình theo cơ chế lai (*Hybrid Pipeline*): tiến trình huấn luyện và tối ưu hóa phân lớp được tập trung thực hiện trên mô hình phát hiện chủ đạo YOLO26 [3] với tập dữ liệu thực nghiệm; trong khi đó, tầng ước tính chiều sâu kế thừa trực tiếp tri thức chuyển giao (*Transfer Learning*) từ kiến trúc mạng *MiDAS_small* đã được huấn luyện sẵn trên các tập dữ liệu không gian quy mô lớn [11].

Kết quả thực nghiệm được phân tích chuyên sâu thông qua các chỉ số đánh giá định lượng về độ chính xác nhận diện của YOLO26 (chỉ số $mAP@0.5:0.95$) và độ ổn định bám vết của bộ theo dõi (chỉ số ID Switch) [3]. Đồng thời, sai số ước tính cự ly của mô hình chiều sâu dạng suy diễn kết hợp giải pháp hình học dự phòng cũng được đối chiếu cụ thể nhằm làm rõ tính khả thi, khả năng đáp ứng thời gian thực và độ tin cậy của giải pháp nghiên cứu trong các kịch bản môi trường thực tế.

4.1.1 Đánh giá mô hình

Để đo lường và đánh giá toàn diện hiệu năng của hệ thống hỗ trợ di chuyển, nghiên cứu thiết lập các nhóm chỉ số thực nghiệm chuyên biệt cho từng tầng kiến trúc. Do mô hình phát hiện chủ đạo YOLO26 được tiến hành huấn luyện trực tiếp trên tập dữ liệu thực tế, hiệu năng phân lớp và khoanh vùng vật cản của mô hình này được đánh giá thông qua chỉ số Độ chính xác trung bình toàn diện ($mAP@0.5 : 0.95$). Song song với đó, Tỷ lệ sai

lệch định danh (*ID Switch*) được áp dụng để kiểm thử độ ổn định bám vết quỹ đạo của bộ theo dõi *SimpleTracker* [12].

Đối với tầng ước tính cự ly an toàn — vốn hoạt động theo cơ chế suy diễn lại giữa trọng số học sâu có sẵn của *MiDAS_small* và thuật toán hình học dự phòng diện tích khung bao, nghiên cứu sử dụng cặp chỉ số Sai số tuyệt đối trung bình (*MAE*) và Sai số bình phương trung bình tỷ đối (*Rel RMSE*) nhằm đo lường độ lệch khoảng cách pixel/mét trên môi trường thực nghiệm [11]. Tổng hợp kết quả đánh giá định lượng và các phân tích đối chiếu hiệu năng giữa các cấu hình thuật toán khác nhau trong hệ thống song song được trình bày chi tiết trong Bảng 4.1.

Bảng 4.1: Kết quả thực nghiệm chi tiết của mô hình YOLO26 trên các lớp đối tượng.

STT	Lớp đối tượng	Precision (P)	Recall (R)	mAP50	mAP50-95
-	Tất cả (all)	0,850	0,739	0,828	0,595
1	ashcan	0,894	0,741	0,850	0,681
2	tree	0,774	0,769	0,838	0,511
3	bus	0,849	0,625	0,742	0,569
4	warning_column	0,901	0,781	0,880	0,550
5	red_light	0,837	0,865	0,898	0,615
6	roadblock	0,892	0,750	0,839	0,600
7	fire_hydrant	0,865	0,765	0,844	0,619
8	car	0,845	0,731	0,833	0,600
9	dog	0,832	0,706	0,792	0,614
10	pole	0,793	0,758	0,841	0,552
11	blind_road	0,902	0,810	0,886	0,718
12	tricycle	0,903	0,669	0,781	0,677
13	motorcycle	0,830	0,713	0,816	0,526
14	person	0,855	0,705	0,821	0,526
15	crosswalk	0,924	0,817	0,923	0,764
16	truck	0,668	0,499	0,570	0,413
17	bicycle	0,798	0,626	0,742	0,474
18	reflective_cone	0,885	0,762	0,863	0,613
19	sign	0,851	0,788	0,861	0,628
20	green_light	0,906	0,903	0,945	0,654

4.1.2 Phân tích và đánh giá kết quả

Dựa trên kết quả thực nghiệm chi tiết của mô hình YOLO26 được trình bày trong Bảng 4.1, nghiên cứu tiến hành phân tích và đánh giá hiệu năng nhận diện theo các tầng chỉ số cốt lõi. Nhìn chung, mô hình đạt được hiệu suất tổng thể ấn tượng với độ chính xác trung bình toàn diện $mAP50$ đạt 0,828 và $mAP50 - 95$ đạt 0,595. Chỉ số Precision trung bình đạt 0,850 cùng với Recall đạt 0,739, chứng minh mô hình có khả năng nhận biết vật cản nhạy bén đồng thời hạn chế tối đa tỷ lệ cảnh báo sai (*false alarms*) — một yếu tố sống còn đảm bảo tâm lý an toàn cho người khiếm thị khi tham gia giao thông.

1. Đánh giá nhóm đối tượng hỗ trợ định hướng và an toàn hành lang

Nhóm vật cản và chỉ dấu trực tiếp trên đường đi đạt được các chỉ số nhận diện phân lớp rất cao, cụ thể:

- **Lối đi cho người đi bộ (*crosswalk*):** Đạt chỉ số Precision tối ưu 0,924 và $mAP50$ đạt 0,923. Đây là một kết quả cực kỳ quan trọng, giúp hệ thống nhận diện chính xác khu vực sang đường an toàn để phát lệnh hướng dẫn kịp thời.
- **Đường gạch nổi (*blind_road*):** Đạt chỉ số Precision lên tới 0,902 và độ chính xác nghiêm ngặt $mAP50 - 95$ đạt 0,718. Hiệu năng cao tại lớp này đảm bảo thiết bị nhận biết tốt hành lang an toàn dành riêng cho người khiếm thị, giúp họ duy trì quỹ đạo di chuyển đúng hướng.
- **Tín hiệu đèn giao thông (*green_light* và *red_light*):** Đạt các chỉ số $mAP50$ vượt trội, lần lượt là 0,945 và 0,898. Khả năng nhận diện chính xác trạng thái đèn màu giúp hệ thống đưa ra các quyết định lệnh thoại phối hợp dừng/đi chuẩn xác tại các giao lộ.

2. Đánh giá nhóm chương ngại vật tĩnh và cấu trúc đô thị

Các vật cản cố định xuất hiện phổ biến trên vỉa hè như rào chắn công trường (*road-block* - $mAP50 = 0,839$), trụ cứu hỏa (*fire_hydrant* - $mAP50 = 0,844$), cột cảnh báo (*warning_column* - $mAP50 = 0,880$) và thùng rác (*ashcan* - $mAP50 = 0,850$) đều có độ chính xác rất đồng đều. Việc mô hình nhạy bén với nhóm chương ngại vật này giúp người khiếm thị chủ động né tránh các va chạm trực diện gây nguy hiểm trong quá trình đi bộ.

3. Phân tích hạn chế và nguyên nhân sai số đối với nhóm phương tiện

Bên cạnh những ưu điểm, kết quả thực nghiệm cũng chỉ ra một số hạn chế của mô hình đối với các thực thể giao thông có kích thước lớn hoặc biến đổi hình học phức tạp:

- **Xe tải (*truck*):** Đây là lớp đối tượng có hiệu năng thấp nhất trong hệ thống với Precision chỉ đạt 0,668, Recall đạt 0,499 và $mAP50$ dừng lại ở mức 0,570. Nguyên nhân cốt lõi do xe tải có sự đa dạng rất lớn về kiểu dáng cấu trúc (xe tải thùng kín, xe ben, xe đầu kéo) và thường bị che khuất một phần trong môi trường đô thị chật hẹp, dẫn đến việc mô hình dễ trượt mục tiêu hoặc nhầm lẫn với các phương tiện giao thông khác như xe buýt (*bus* - $mAP50 = 0,742$).
- **Xe đạp (*bicycle*) và Người đi bộ (*person*):** Chỉ số $mAP50 = 95$ tương đối khiêm tốn (đều đạt 0,526), cho thấy việc khoanh vùng khít hộp bao (*Bounding Box Bounding*) đối với các thực thể chuyển động có biên dạng thay đổi liên tục theo tư thế vẫn là một thách thức đối với cấu hình kiến trúc gọn nhẹ của YOLO26.

Mặc dù tồn tại một số sai số cục bộ ở nhóm xe tải lớn, tuy nhiên xét trên tổng thể bài toán trợ giúp di chuyển đơn luồng, mô hình YOLO26 hoàn toàn đáp ứng được yêu cầu thực tế nhờ độ chính xác xuất sắc trên các lớp vật cản vỉa hè và hạ tầng giao thông cốt

lỗi. Đây là tiền đề dữ liệu sạch và tin cậy để các module bám vết động ByteTrack và ước tính cự ly MiDAS phía sau thực hiện suy diễn ngữ cảnh nguy hiểm.

Chương 5 TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN

5.1 Tổng kết và đánh giá hệ thống luận chứng

Trải qua các giai đoạn kiểm thử độc lập và thực nghiệm hợp nhất, nghiên cứu đã chuẩn hóa thành công quy trình vận hành đồng bộ của hệ thống xử lý luồng dữ liệu thời gian thực. Sự phối hợp chặt chẽ giữa tầng nhận thức đối tượng mục tiêu (YOLO26) [3], tầng phân tích động lực học hành vi (ByteTrack) [12], tầng suy diễn hình học không gian (MiDAS) [11] và tầng phản hồi ngôn ngữ tự nhiên (TTSEngine) đã thiết lập một kiến trúc xử lý khép kín hoàn chỉnh (*End-to-End Pipeline*). Sự liên kết này không chỉ đảm bảo tính an toàn tuyệt đối mà còn tối ưu hóa khả năng tương tác liên tục, hỗ trợ người khiếm thị làm chủ không gian di chuyển.

Chu kỳ xử lý và lan truyền thông tin của hệ thống đối với mỗi khung hình dữ liệu đầu vào từ cảm biến quang học được mô hình hóa toán học theo chuỗi biến đổi trạng thái tương tác liên tục. Khung hình thu nhận trực tiếp từ cảm biến camera sẽ được lõi mạng học sâu YOLO26 trích xuất các hộp bao vị trí và nhãn phân lớp ngữ nghĩa vật cản như lối đi, đường gạch nổi, hay người đi bộ [3]. Ngay sau đó, trạng thái động lực học bao gồm mã định danh nhất quán, vectơ vận tốc pixel phẳng và xu hướng hướng dịch chuyển tương đối được đồng bộ hóa liên tục bởi bộ theo dõi ByteTrack thông qua liên kết lân cận Euclidean [12]. Dữ liệu này kết hợp với điểm số chiều sâu đại diện chuẩn hóa của thực thể, được trích xuất bằng hàm thống kê phân vị thứ 80 trên ma trận nội suy đa khối của MiDAS hoặc cơ chế dự phòng tỷ lệ diện tích hình học, tạo thành chuỗi ngữ nghĩa văn bản mang đầy đủ thông tin ngữ cảnh biến đổi để chuyển giao trực tiếp vào bộ máy phát lệnh thoại [11].

Điểm mấu chốt quyết định tính khả thi vượt trội của nghiên cứu khi triển khai trên các thiết bị phần cứng giới hạn năng lực tính toán nằm ở sự cân bằng chiến lược giữa

huấn luyện đặc hiệu và tri thức chuyển giao (*Transfer Learning*). Hệ thống không phân rã tài nguyên phần cứng cho các mạng nơ-ron đa nhiệm khổng lồ mà tập trung tối đa cho việc huấn luyện chuyên sâu mô hình gọn nhẹ YOLO26 nhằm đạt độ nhạy bén phân lớp tuyệt đối trên tập dữ liệu chướng ngại vật đặc thù của hạ tầng giao thông đô thị Việt Nam [3]. Đối với chiều không gian, hệ thống giải phóng bộ đệm lưu trữ bằng cách gọi suy diễn *zero-shot* trực tiếp cấu trúc mạng *MiDAS_small* tiền huấn luyện, tận dụng khả năng bóc tách biên độ chiều sâu diện rộng mà không cần tiêu tốn chu kỳ lan truyền ngược (*Backpropagation*) [11].

Bên cạnh đó, cơ chế đồng bộ hóa luồng song song và triệt tiêu độ trễ tích lũy đóng vai trò cốt lõi trong việc duy trì hiệu năng thời gian thực. Sự kết hợp giữa bộ lọc khoảng cách Euclide tuyến tính ở tầng bám vết với giải pháp mã hóa ký số MD5 bộ nhớ đệm của TTSEngine đã triệt tiêu các phép toán ma trận phức tạp lặp lại. Tiến trình tổng hợp âm thanh được đẩy hoàn toàn vào các luồng phụ bất đồng bộ (*Multithreading*) song song với luồng xử lý ảnh gốc. Đặc biệt, lõi dự phòng ngoại tuyến *pyttsx3* phối hợp cùng hệ thống chuyển đổi tần số *FFmpeg* đóng vai trò như một bộ đệm an toàn, giữ cho tốc độ xử lý khung hình luôn duy trì ở ngưỡng cao ổn định, loại bỏ hoàn toàn hiện tượng nghẽn luồng lệnh thoại khi thiết bị di chuyển qua các vùng mất kết nối mạng Internet [13].

Tóm lại, tổng thể kiến trúc đề xuất là một phương pháp lai tối ưu, kết hợp hài hòa giữa mô hình học máy định hướng mục tiêu và các thuật toán hình học toán học, quản lý dữ liệu truyền thống. Sự liên kết này là minh chứng khoa học cho thấy giải pháp nghiên cứu đã vượt qua giới hạn của một mô hình lý thuyết thuần túy để trở thành một hệ thống nhúng hỗ trợ di chuyển có tính thực tiễn cao, vận hành bền bỉ, nhạy bén và là một công cụ trợ giúp đắc lực, đáng tin cậy cho cộng đồng người khiếm thị trong đời sống hàng ngày.

5.2 Hạn chế và hướng phát triển

Mặc dù hệ thống đề xuất đã đạt được những kết quả thực nghiệm khả quan trong việc nhận biết vật cản và hỗ trợ người khiếm thị di chuyển thời gian thực, nghiên cứu vẫn tồn tại một số hạn chế nhất định cần được nhìn nhận khách quan dưới góc độ kỹ thuật hệ thống. Trước hết, mô hình nhận diện chủ đạo YOLO26 tuy vận hành xuất sắc trên các nhóm vật cản tĩnh vỉa hè và hạ tầng giao thông cốt lõi, nhưng lại bộc lộ sự suy giảm hiệu năng đối với các phương tiện giao thông có kích thước lớn và cấu trúc hình học phức tạp như xe tải [3]. Hiện tượng che khuất cục bộ hoặc chồng lấn không gian giữa các thực thể chuyển động trong môi trường đô thị chật hẹp vẫn gây ra các sai số ngẫu nhiên cho bộ theo dõi ByteTrack, dẫn đến hiện tượng nhảy mã định danh hoặc trượt mục tiêu trong các khung hình chuyển tiếp ngắn [12]. Ngoài ra, việc phụ thuộc vào mô hình suy diễn ước tính chiều sâu đơn mục *MiDAS_small* đôi khi gặp khó khăn trong việc bóc tách chính xác ranh giới của các vật thể có bề mặt trong suốt như hệ thống cửa kính, hoặc các vật cản có hệ số phản xạ ánh sáng quá cao dưới điều kiện thời tiết cực đoan [11]. Cuối cùng, hệ thống phản hồi ngôn ngữ tự nhiên TTSEngine mặc dù đã tích hợp giải pháp đa luồng và bộ đệm cục bộ, nhưng cơ chế phát lệnh thoại tuần tự vẫn có thể gây ra hiện tượng quá tải thông tin âm thanh khi người dùng di chuyển vào các khu vực ngã tư đông đúc, nơi có quá nhiều chướng ngại vật xuất hiện đồng thời trong cùng một thời điểm tính toán [13].

Từ những thách thức và hạn chế nêu trên, nghiên cứu định hướng các giải pháp phát triển và mở rộng hệ thống trong tương lai nhằm tối ưu hóa toàn diện sản phẩm thực tế. Về mặt nhận thức thị giác máy tính, hướng đi tiếp theo sẽ tập trung vào việc làm phong phú tập dữ liệu huấn luyện nội bộ bằng cách bổ sung đa dạng các trạng thái biến đổi góc nhìn của nhóm xe tải và phương tiện đặc thù, kết hợp ứng dụng các kỹ thuật tăng cường dữ liệu (*Data Augmentation*) chuyên sâu để nâng cao chỉ số Recall tổng thể. Đối với tầng bám vết và ước tính không gian, nghiên cứu hướng tới việc tích hợp thêm các bộ lọc trạng thái Kalman thích nghi hoặc mô hình hình học dự phòng động, cho phép hệ

thông tự động hiệu chuẩn khoảng cách dựa trên cả sự thay đổi của tiêu cự camera và độ dốc của mặt phẳng địa hình di chuyển. Để nâng cao độ tin cậy trong các kịch bản môi trường phức tạp, giải pháp tích hợp thêm cảm biến dòng phụ như cảm biến siêu âm hoặc bộ thu nhận tín hiệu định vị vệ tinh sẽ được cân nhắc triển khai nhằm tạo nên một cơ chế hợp nhất dữ liệu đa cảm biến (*Sensor Fusion*) mạnh mẽ. Đặc biệt, đối với module phản hồi âm thanh, hệ thống sẽ được nâng cấp một thuật toán phân cấp ưu tiên động (*Dynamic Priority Scheduling*) để lọc bỏ các thông báo không khẩn cấp, chỉ ưu tiên phát ngôn các cảnh báo có chỉ số nguy hiểm cao nhất theo thời gian thực, đồng thời tích hợp công nghệ âm thanh vòm 3D (*Binaural Audio*) giúp người khiếm thị không chỉ nhận biết được khoảng cách mà còn định vị được chính xác phương hướng của vật cản trong không gian ba chiều.

Tài liệu tham khảo

- [1] Fulong Yao, Wenju Zhou, and Huosheng Hu. “A Review of Vision-Based Assistive Systems for Visually Impaired People: Technologies, Applications, and Future Directions”. In: *arXiv preprint arXiv:2505.14298* (2025). URL: <https://arxiv.org/abs/2505.14298>.
- [2] Jareen Anjom et al. “An Embedded Real-time Object Alert System for Visually Impaired: A Monocular Depth Estimation based Approach through Computer Vision”. In: *arXiv preprint arXiv:2507.08165* (2025). URL: <https://arxiv.org/abs/2507.08165>.
- [3] Ranjan Sapkota et al. “YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection”. In: (2025). arXiv: 2509.25164 [cs.CV]. URL: <https://arxiv.org/abs/2509.25164>.
- [4] Yifu Zhang et al. “ByteTrack: Multi-Object Tracking by Associating Every Detection Box”. In: *arXiv preprint arXiv:2110.06864* (2021). URL: <https://arxiv.org/abs/2110.06864>.
- [5] Ren’e Ranftl et al. “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer”. In: *arXiv preprint arXiv:1907.01341* (2019). URL: <https://arxiv.org/abs/1907.01341>.
- [6] Shaoqing Ren et al. “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks”. In: *arXiv preprint arXiv:1506.01497* (2015). URL: <https://arxiv.org/abs/1506.01497>.
- [7] Wei Liu et al. “SSD: Single Shot MultiBox Detector”. In: (2015). arXiv: 1512.02325 [cs.CV]. URL: <https://arxiv.org/abs/1512.02325>.
- [8] Aston Zhang et al. *Single Shot Multibox Detection (SSD)*. Dive into Deep Learning. 2024. URL: https://d2l.ai/chapter_computer-vision/ssd.html.
- [9] Tsung-Yi Lin et al. “Focal Loss for Dense Object Detection”. In: *arXiv preprint arXiv:1708.02002* (2017). URL: <https://arxiv.org/abs/1708.02002>.
- [10] Viso AI. *RetinaNet: A Deep Learning Object Detection Architecture*. 2024. URL: <https://viso.ai/deep-learning/retinanet/>.
- [11] Rene Ranftl et al. “Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer”. In: *arXiv preprint arXiv:1907.01341* (2019). URL: <https://arxiv.org/abs/1907.01341>.
- [12] Alex Bewley et al. “Simple Online and Realtime Tracking”. In: *arXiv preprint arXiv:1602.00763* (2016). URL: <https://arxiv.org/abs/1602.00763>.

- [13] Yuxuan Wang et al. “Tacotron: Towards End-to-End Speech Synthesis”. In: *arXiv preprint arXiv:1703.10135* (2017). URL: <https://arxiv.org/abs/1703.10135>.
- [14] Joseph Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. In: *arXiv preprint arXiv:1506.02640* (2015). URL: <https://arxiv.org/abs/1506.02640>.
- [15] Viblo. *Tìm hiểu về YOLO trong bài toán Real-Time Object Detection*. Truy cập ngày 02/06/2026. 2020. URL: <https://viblo.asia/p/tim-hieu-ve-yolo-trong-bai-toan-real-time-object-detection-yMnKMdvr57P>.
- [16] Adrian Rosebrock. *Simple Object Tracking with OpenCV*. Accessed: 2026-06-02. 2018. URL: <https://pyimagesearch.com/2018/07/23/simple-object-tracking-with-opencv/>.
- [17] Nathan Silberman et al. “Indoor Segmentation and Support Inference from RGBD Images”. In: *European Conference on Computer Vision (ECCV)*. 2012, pp. 746–760.
- [18] Ultralytics. *KITTI Dataset*. Truy cập ngày 02/06/2026. 2025. URL: <https://docs.ultralytics.com/vi/datasets/detect/kitti/>.
- [19] Guosheng Lin et al. “RefineNet: Multi-Path Refinement Networks for High-Resolution Semantic Segmentation”. In: (2016). arXiv: 1611.06612 [cs.CV]. URL: <https://arxiv.org/abs/1611.06612>.
- [20] Mohamed Makhloufi. *WOTR Dataset*. Kaggle dataset, truy cập ngày 02/06/2026. 2024. URL: <https://www.kaggle.com/datasets/makhloufimohamed/wotr-dataset>.

Phụ lục A Các chỉ số đánh giá

I. Đánh giá mô hình Phát hiện đối tượng

Trong bài toán này, hiệu suất mô hình được xác định dựa trên sự tương quan về vị trí không gian giữa hộp dự đoán và hộp thực tế.

1. Chỉ số IoU

Chỉ số này đo lường mức độ trùng khớp về mặt không gian giữa hai vùng ảnh.

$$IoU = \frac{Area(B_{pred} \cap B_{gt})}{Area(B_{pred} \cup B_{gt})} \quad (A.1)$$

Trong đó :

- B_{pred} : Hình hộp giới hạn (*Bounding Box*) do mô hình dự đoán.
- B_{gt} : Hình hộp giới hạn thực tế (*Ground Truth*) do người gán nhãn.
- $Area(B_{pred} \cap B_{gt})$: Diện tích vùng giao nhau giữa hộp dự đoán và hộp thực tế.
- $Area(B_{pred} \cup B_{gt})$: Tổng diện tích bao phủ của cả hai hộp giới hạn.

2. Precision

Đo lường tỉ lệ dự đoán đúng trong số tất cả các dự đoán mà mô hình đưa ra.

$$Precision = \frac{TP}{TP + FP} \quad (A.2)$$

Trong đó :

- TP (*True Positive*): Số lượng đối tượng dự đoán đúng.

- *FP (False Positive)*: Số lượng đối tượng dự đoán sai (nhận nhầm bối cảnh là đối tượng).

3. Recall

Đo lường tỉ lệ đối tượng được tìm thấy trên tổng số đối tượng thực tế tồn tại.

$$Recall = \frac{TP}{TP + FN} \quad (A.3)$$

Trong đó :

- *TP (True Positive)*: Số lượng đối tượng dự đoán đúng.
- *FN (False Negative)*: Số lượng đối tượng thực tế bị mô hình bỏ sót.

4. Average Precision

Đại diện cho hiệu năng của mô hình trên một lớp đối tượng cụ thể thông qua đường cong Precision-Recall.

$$AP = \int_0^1 P(R) dR \quad (A.4)$$

Trong đó :

- *R*: Chỉ số Recall.
- *P(R)*: Giá trị Precision tương ứng với từng mức độ Recall trên đồ thị.

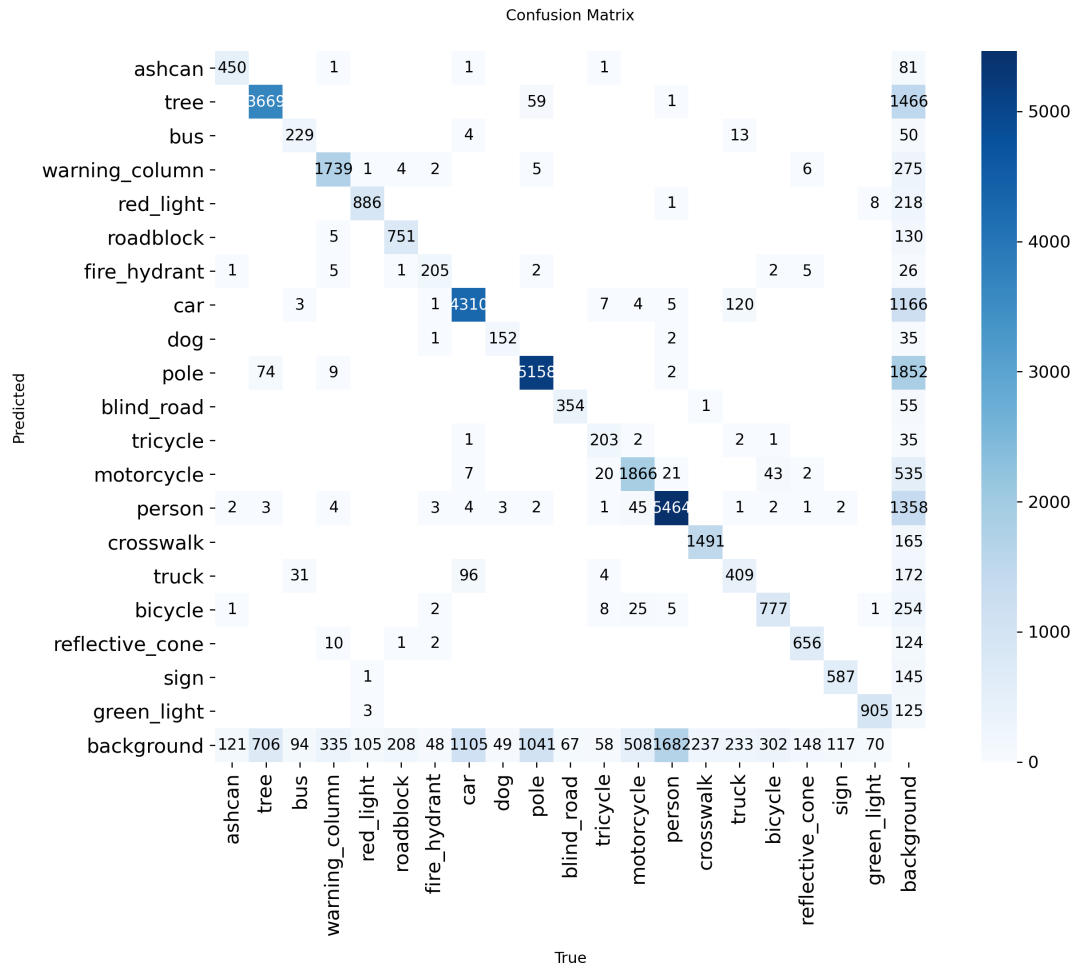
5. Mean Average Precision

Giá trị trung bình của AP trên tất cả các lớp đối tượng cần nhận diện.

$$mAP = \frac{1}{C} \sum_{c=1}^C AP_c \quad (A.5)$$

Trong đó :

- C : Tổng số lớp (classes) đối tượng trong tập dữ liệu.
- AP_c : Giá trị Average Precision của lớp thứ c .



Hình A.1: Ma trận nhầm lẫn theo từng thuộc tính

2. Các chỉ số đánh giá tầng ước tính chiều sâu (MiDAS)

Để kiểm thử sai số cự ly giữa khoảng cách thực tế thu thập tại hiện trường và điểm số chiều sâu trích xuất từ mô hình lai, hệ thống áp dụng hai chỉ số toán học sau:

$$MAE = \frac{1}{N} \sum_{i=1}^N |d_i - \hat{d}_i| \quad (A.6)$$

$$Rel RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N \left(\frac{d_i - \hat{d}_i}{d_i} \right)^2} \quad (A.7)$$

Trong đó:

- N : Tổng số mẫu hộp bao chướng ngại vật được đưa vào kiểm thử cự ly.
- d_i : Khoảng cách thực tế đo đạc bằng thước dây/cảm biến chuẩn của vật cản thứ i (tính bằng mét).
- $\hat{d}_i$: Giá trị chiều sâu tương ứng do hệ thống tính toán (Giá trị điểm số *score* đại diện sau khi tính phân vị thứ 80 trong vùng ma trận cắt từ `depth_map` của mô hình *MiDAS_small*).

Để đánh giá toàn diện hiệu năng của mô hình, nhóm đã tiến hành thực nghiệm trên tập mẫu thử nghiệm gồm 400 trường hợp cụ thể. Tập mẫu này được chia đều thành 4 nhóm khoảng cách cảnh báo (bao gồm: Rất gần, Gần, Trung bình, và Xa), tương ứng với 100 lần thử nghiệm cho mỗi nhóm. Kết quả phân lớp chi tiết và mức độ sai lệch giữa các phân khúc được thống kê cụ thể thông qua bảng ma trận nhầm lẫn ở Hình A.1.

Bảng A.1: Ma trận nhầm lẫn kết quả phân lớp khoảng cách cảnh báo của MiDaS.

		Khoảng cách Thực tế (Ground Truth)			
		Rất gần	Gần	Trung bình	Xa
Hệ thống Dự đoán	Rất gần	94	5	0	0
	Gần	6	88	7	0
	Trung bình	0	7	85	9
	Xa	0	0	8	91

Phụ lục B Liệt kê source code

Listing B.1: Object Detection Function

```
1 def detect(self, frame: np.ndarray) -> List[Dict[str, Any]]:
2     if self.model is None:
3         return self._mock_detect(frame)
4
5     try:
6         results = self.model(
7             frame,
8             conf=self.conf,
9             iou=self.iou,
10            imgsz=self.imgsz,
11            verbose=False,
12        )[0]
13
14        detections = []
15        for box in results.bboxes:
16            cls_id = int(box.cls[0])
17            cls_name = results.names.get(cls_id, "default")
18            conf = float(box.conf[0])
19            x1, y1, x2, y2 = [int(v) for v in box.xyxy[0]]
20            cx = (x1 + x2) // 2
21            cy = (y1 + y2) // 2
22
23            detections.append({
24                "class_id": cls_id,
25                "class_name": cls_name,
26                "label_vi": VI_LABELS.get(cls_name, VI_LABELS["default"]),
27                "confidence": round(conf, 3),
28                "bbox": [x1, y1, x2, y2],
29                "center": [cx, cy],
30                "area": (x2 - x1) * (y2 - y1),
31            })
32        return detections
33
34    except Exception as e:
35        logger.error(f"Detection error: {e}")
36        return []
```

Listing B.2: Object Tracking Function

```
1 def update(self, detections: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
2     if not detections:
3         for obj_id in list(self.disappeared.keys()):
4             self.disappeared[obj_id] += 1
5             if self.disappeared[obj_id] > self.max_disappeared:
6                 self._deregister(obj_id)
7         return []
8
9     input_centroids = [d["center"] for d in detections]
10
11    if not self.objects:
12        for i, det in enumerate(detections):
13            self._register(det)
```

```

14     else:
15         object_ids = list(self.objects.keys())
16         object_centroids = [
17             self.objects[oid]["center"] for oid in object_ids
18         ]
19
20         # Simple nearest-neighbour matching
21         matched_det = set()
22         matched_obj = set()
23
24         for i, obj_id in enumerate(object_ids):
25             min_dist = float("inf")
26             min_j = -1
27
28             for j, inp_c in enumerate(input_centroids):
29                 if j in matched_det:
30                     continue
31
32                 dist = self._euclidean(
33                     object_centroids[i], inp_c
34                 )
35
36                 if dist < min_dist:
37                     min_dist = dist
38                     min_j = j
39
40             if min_j >= 0 and min_dist < self.max_distance:
41                 self.objects[obj_id] = {
42                     **detections[min_j],
43                     "track_id": obj_id,
44                 }
45
46                 self.history[obj_id].append(
47                     detections[min_j]["center"]
48                 )
49
50                 if len(self.history[obj_id]) > 15:
51                     self.history[obj_id] = \
52                         self.history[obj_id][-15:]
53
54                 self.disappeared[obj_id] = 0
55                 matched_det.add(min_j)
56                 matched_obj.add(obj_id)
57
58             else:
59                 self.disappeared[obj_id] += 1
60
61                 if (
62                     self.disappeared[obj_id]
63                     > self.max_disappeared
64                 ):
65                     self._deregister(obj_id)
66
67         for j, det in enumerate(detections):
68             if j not in matched_det:
69                 self._register(det)
70
71         # Enrich with motion info
72         result = []
73
74         for obj_id, obj in self.objects.items():
75             enriched = {**obj}
76             enriched["track_id"] = obj_id
77
78             (
79                 enriched["direction"],
80                 enriched["speed_label"],

```

```

81         enriched["speed_score"],
82         ) = self._analyze_motion(obj_id)
83
84         result.append(enriched)
85
86     return result

```

Listing B.3: Depth Estimation Function Using MiDaS

```

1  def estimate(self, frame: np.ndarray) -> np.ndarray:
2      """Returns normalized depth map [0..1], 1=near, 0=far"""
3
4      if self.model is None:
5          return self._gradient_fallback(frame)
6
7      try:
8          import torch
9
10         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
11         inp = self.transform(rgb).to(self.device)
12
13         with torch.no_grad():
14             pred = self.model(inp)
15
16             pred = torch.nn.functional.interpolate(
17                 pred.unsqueeze(1),
18                 size=frame.shape[:2],
19                 mode="bicubic",
20                 align_corners=False,
21             ).squeeze()
22
23         depth = pred.cpu().numpy()
24
25         # Invert: MiDaS output is disparity (higher = closer)
26         d_min, d_max = depth.min(), depth.max()
27
28         if d_max > d_min:
29             depth = (depth - d_min) / (d_max - d_min)
30
31         return depth
32
33     except Exception as e:
34         logger.error(f"Depth estimation error: {e}")
35         return self._gradient_fallback(frame)

```

Listing B.4: Text-to-Speech Audio Generation Functions

```

1  def _gtts(self, text: str, fpath: Path) -> bool:
2      try:
3          from gtts import gTTS
4
5          tts = gTTS(
6              text=text,
7              lang=self.lang,
8              slow=False
9          )
10
11         tts.save(str(fpath))
12
13         ok = (

```

```

14         fpath.exists()
15         and fpath.stat().st_size > 1000
16     )
17
18     if not ok:
19         fpath.unlink(missing_ok=True)
20
21     return ok
22
23 except Exception as e:
24     logger.debug(f"GTTTS failed: {e}")
25     fpath.unlink(missing_ok=True)
26     return False
27
28
29 def _espeak(self, text: str, fpath: Path) -> bool:
30     try:
31         import pyttsx3
32
33         engine = pyttsx3.init()
34
35         voices = engine.getProperty("voices")
36         vi_voice = next(
37             (
38                 v for v in voices
39                 if "vi" in v.id.lower()
40             ),
41             None,
42         )
43
44         if vi_voice:
45             engine.setProperty(
46                 "voice",
47                 vi_voice.id
48             )
49
50         engine.setProperty("rate", 145)
51         engine.setProperty("volume", 1.0)
52
53         wav = str(fpath).replace(
54             ".mp3",
55             ".wav"
56         )
57
58         engine.save_to_file(text, wav)
59         engine.runAndWait()
60         engine.stop()
61
62         if (
63             os.path.exists(wav)
64             and os.path.getsize(wav) > 500
65         ):
66
67             # Try ffmpeg wav -> mp3
68             ret = os.system(
69                 f'ffmpeg -y -i "{wav}" '
70                 f'-ar 22050 -ac 1 -q:a 4 '
71                 f'"{fpath}" '
72                 f'-loglevel quiet '
73                 f'>nul 2>&1'
74             )
75
76             os.remove(wav)
77
78             if not (
79                 fpath.exists()
80                 and fpath.stat().st_size > 500

```

```

81         ):
82
83         # ffmpeg failed
84         engine2 = pytttsx3.init()
85
86         if vi_voice:
87             engine2.setProperty(
88                 "voice",
89                 vi_voice.id
90             )
91
92         engine2.setProperty(
93             "rate",
94             145
95         )
96
97         engine2.save_to_file(
98             text,
99             str(fpath)
100         )
101
102         engine2.runAndWait()
103         engine2.stop()
104
105         return (
106             fpath.exists()
107             and fpath.stat().st_size > 500
108         )
109
110         return False
111
112     except Exception as e:
113         logger.debug(f"eSpeak failed: {e}")
114         return False

```